

Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs (Problems and Solutions)

D. Results

E. Conditional and Stacked GANs

F. Evaluating GANs

Motivation

Motivation: endowing computers with an understanding of our world.

Goal: collect a lot of data, use it to train a model to generate similar data from scratch.

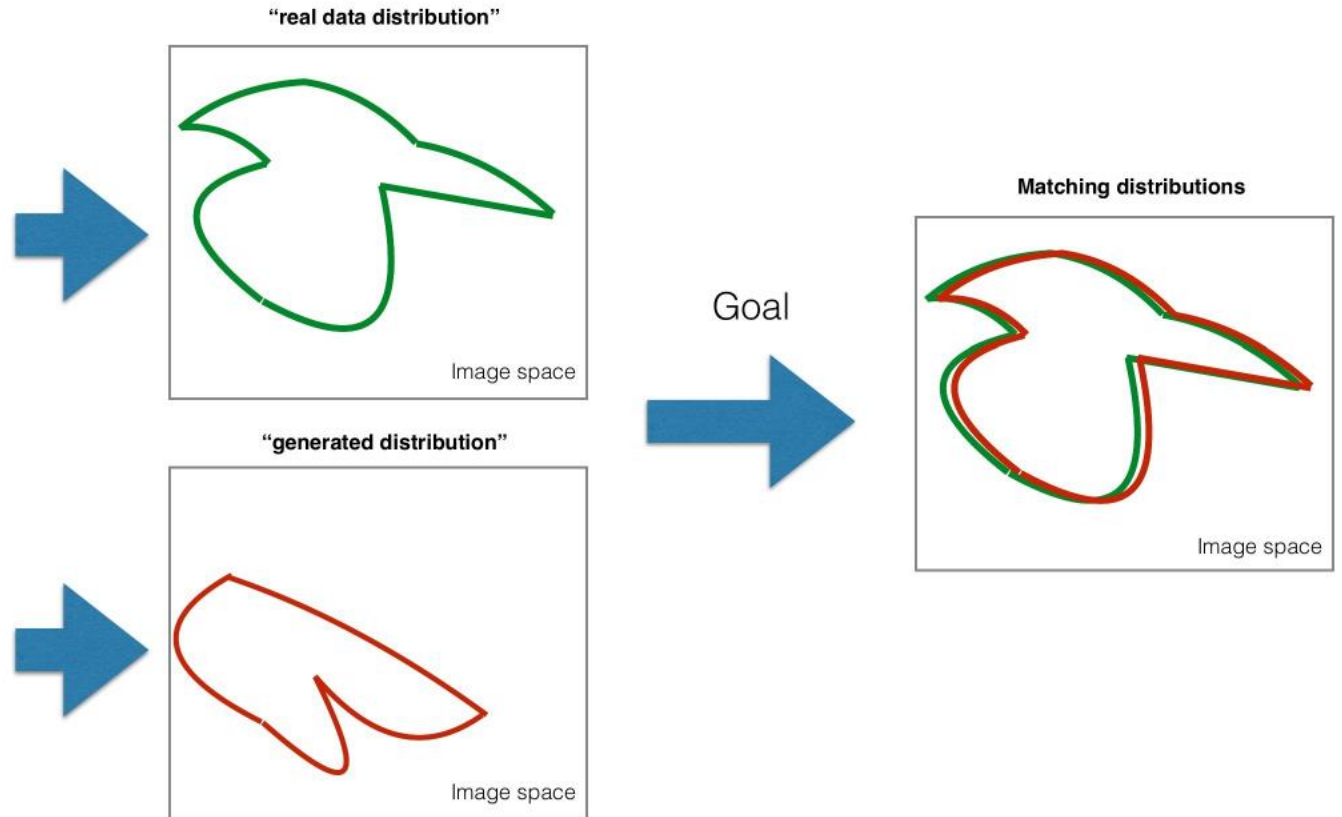
Motivation

Probability distributions:

Samples from the “real data distribution”



Samples from the “generated distribution”

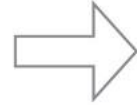
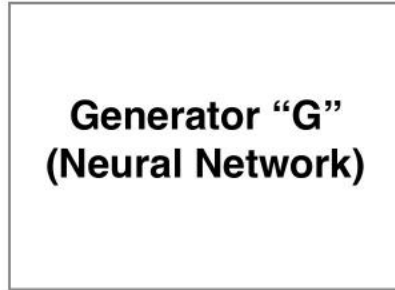
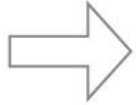


Generator / Discriminator (G/D) Game

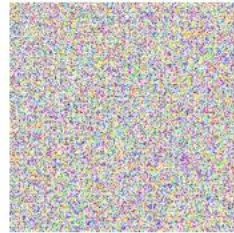
100-d
random code

$$\begin{pmatrix} 0.47 \\ \vdots \\ 0.19 \end{pmatrix}$$

z



(64,64,3)
generated image



$\neq$



How can we train G to generate images from the true data distributions?

G/D Game

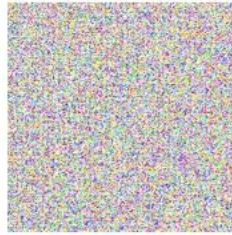
100-d
random code

$$\begin{pmatrix} 0.47 \\ \vdots \\ 0.19 \end{pmatrix}$$

z

**Generator “G”
(Neural Network)**

(64,64,3)
generated image



**Run Adam simultaneously on two minibatches
(true data / generated data)**

x

**Discriminator “D”
(Neural Network)**

Gradients

Binary classification

$$\begin{cases} y = 0 & \text{if } x = G(z) \\ y = 1 & \text{otherwise} \end{cases}$$

**Real images
(database)**



Probability distributions

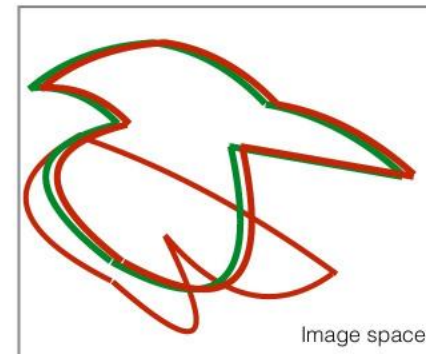


Image space

G/D Game

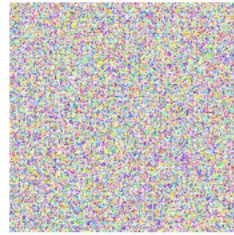
100-d
random code

$$\begin{pmatrix} 0.47 \\ \vdots \\ 0.19 \end{pmatrix}$$

z

**Generator “G”
(Neural Network)**

(64,64,3)
generated image



**End goal: G is outputting
images that are
indistinguishable from real
images for D**

**Real images
(database)**



x

**Discriminator “D”
(Neural Network)**

Gradients Binary classification

$$\begin{cases} y = 0 & \text{if } x = G(z) \\ y = 1 & \text{otherwise} \end{cases}$$

Probability distribution

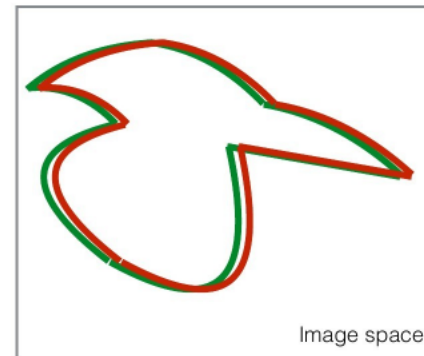


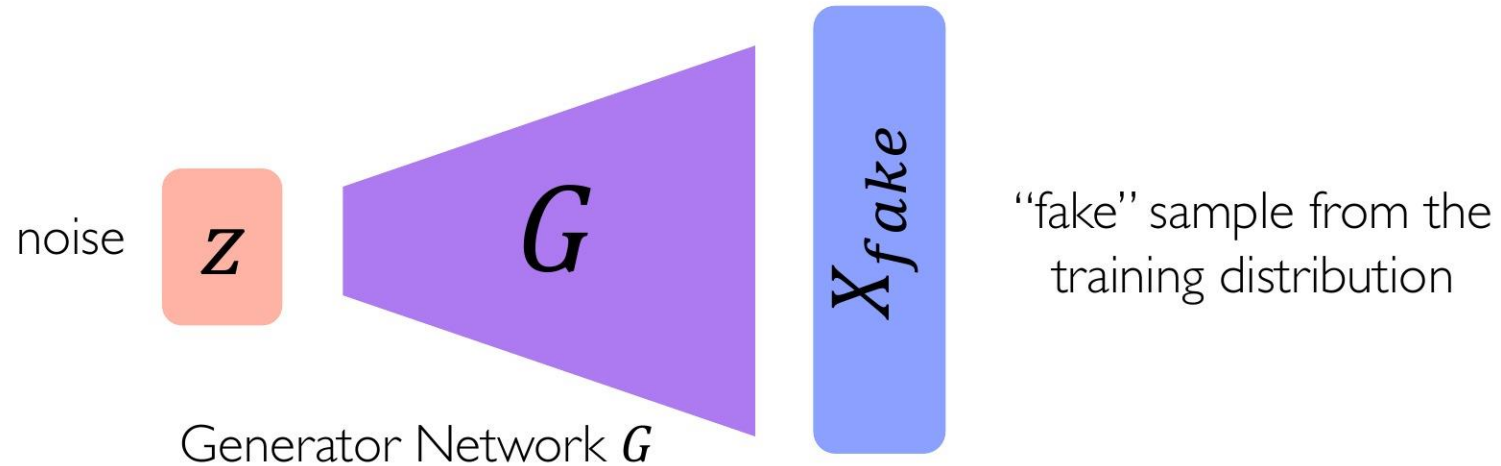
Image space

What if we just want to sample?

Idea: don't explicitly model density, and instead just sample to generate new instances.

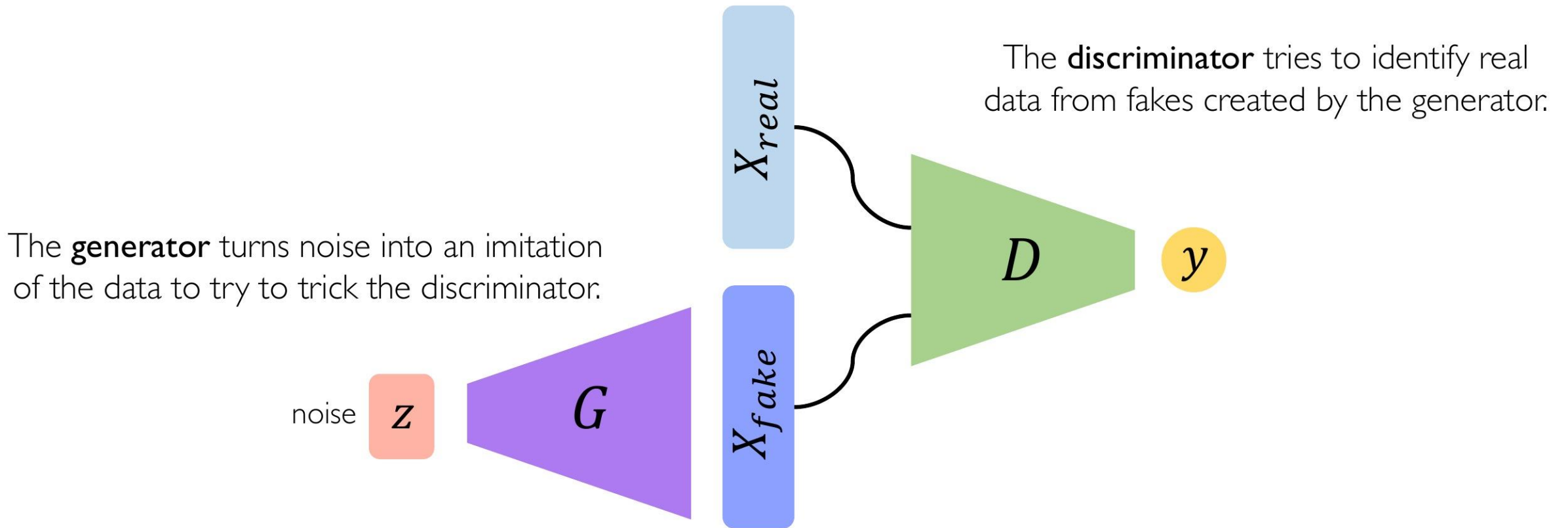
Problem: want to sample from complex distribution – can't do this directly!

Solution: sample from something simple (noise), learn a transformation to the training distribution.



Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a way to make a generative model by having two neural networks compete with each other.



Intuition behind GANs

Generator starts from noise to try to create an imitation of the data.

Generator



 Fake data

Intuition behind GANs

Discriminator looks at both real data and fake data created by the generator.

Discriminator

Generator



 Fake data

Intuition behind GANs

Discriminator looks at both real data and fake data created by the generator.

Discriminator

Generator

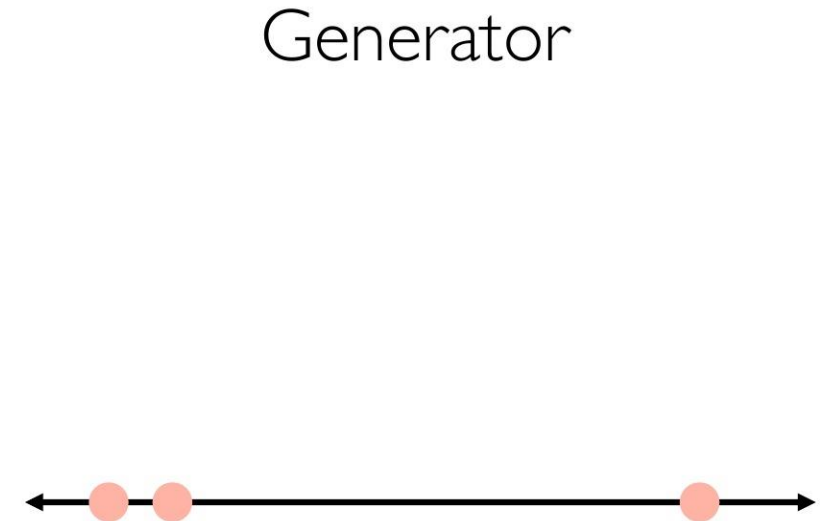
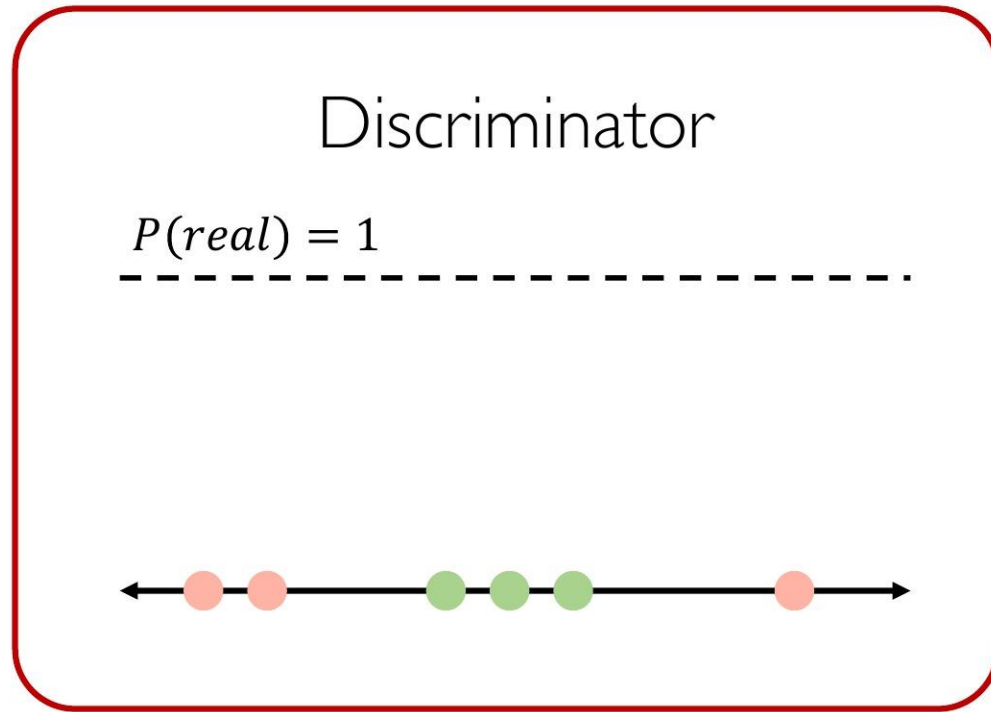


 Real data

 Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

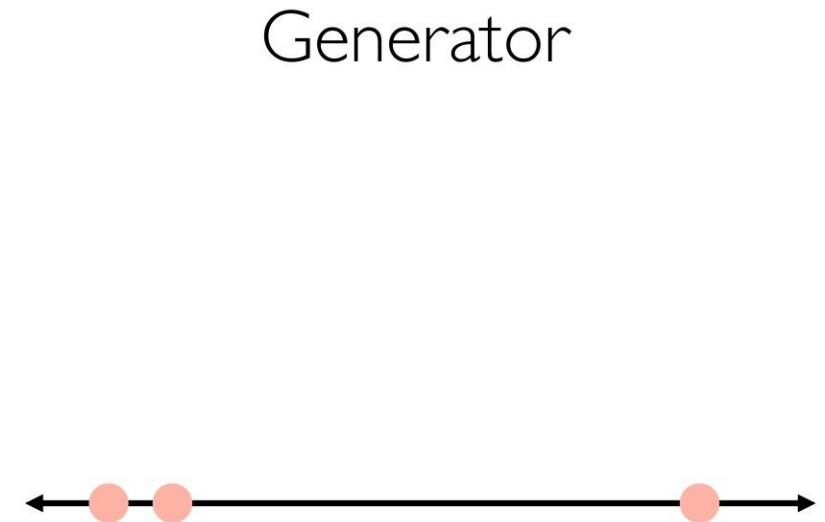
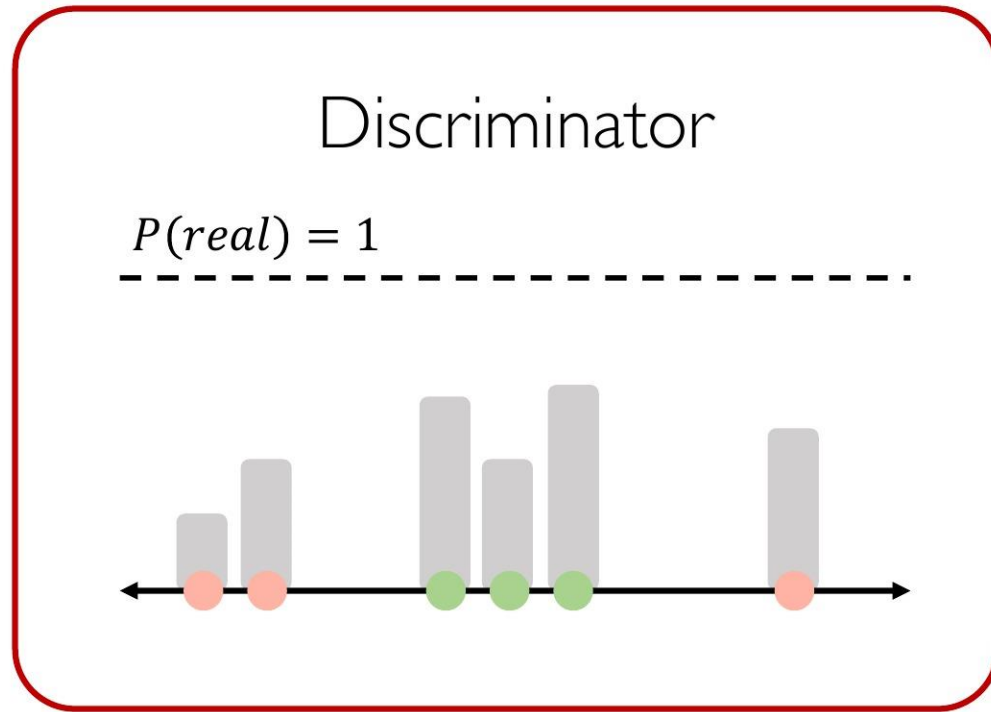


 Real data

 Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

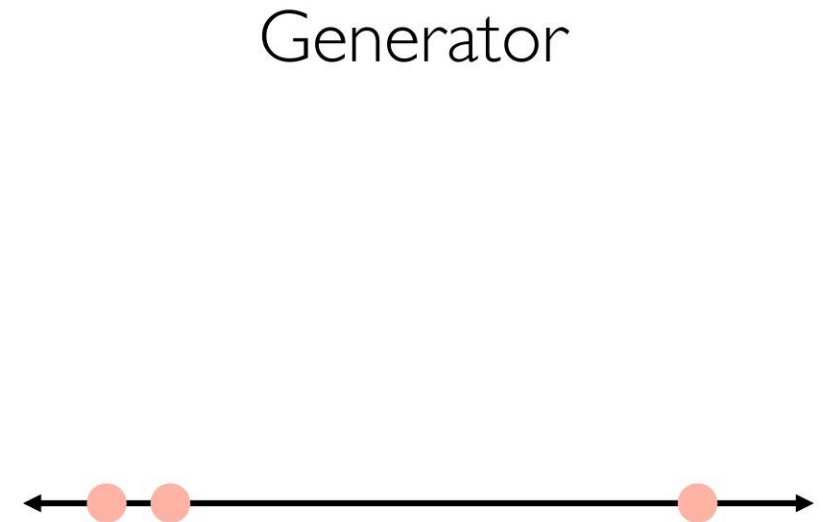
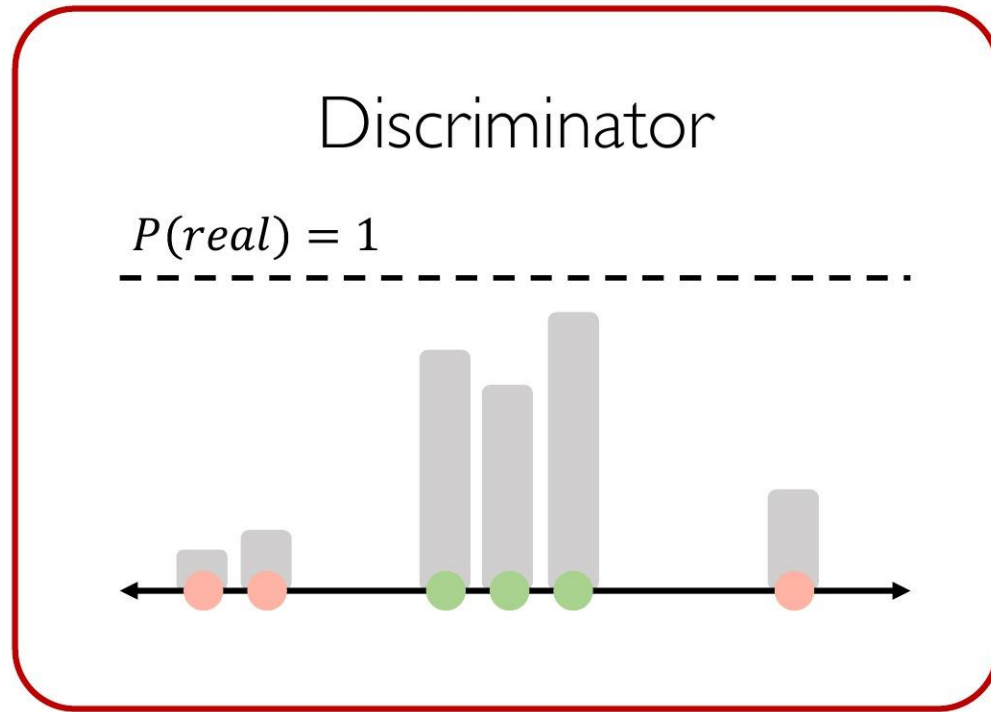


● Real data

● Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

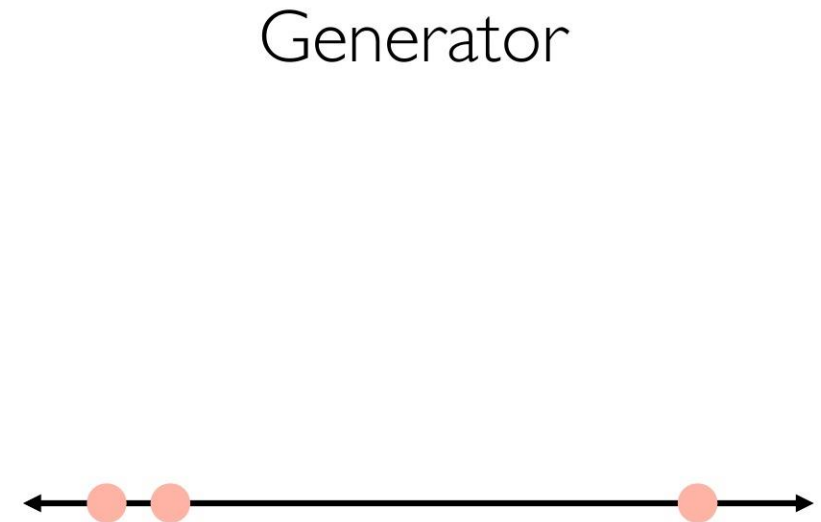
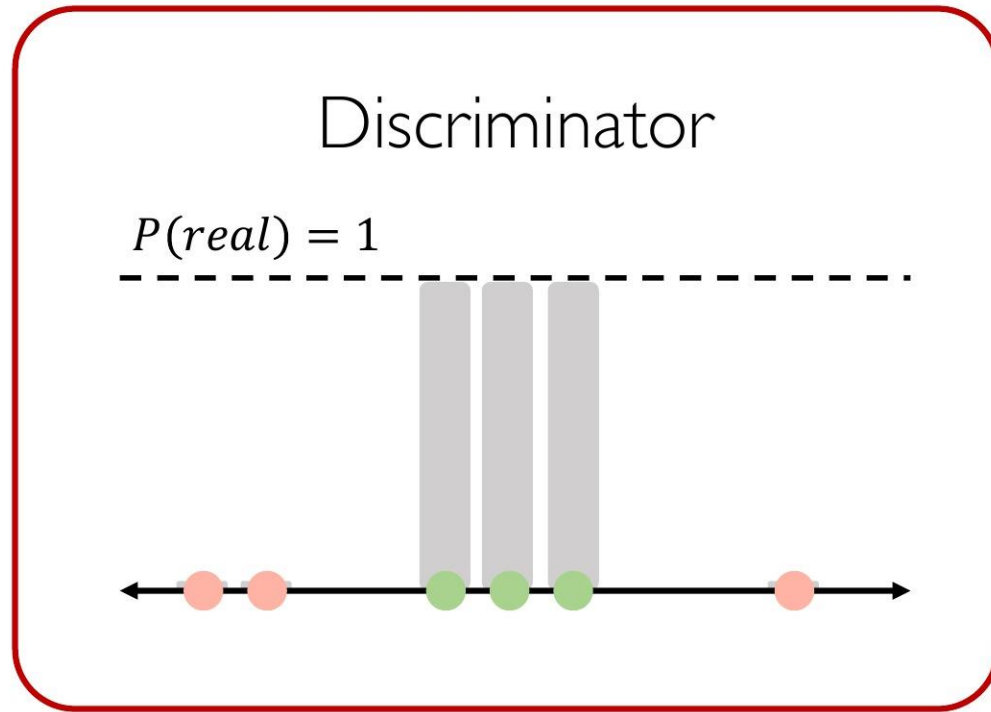


● Real data

● Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

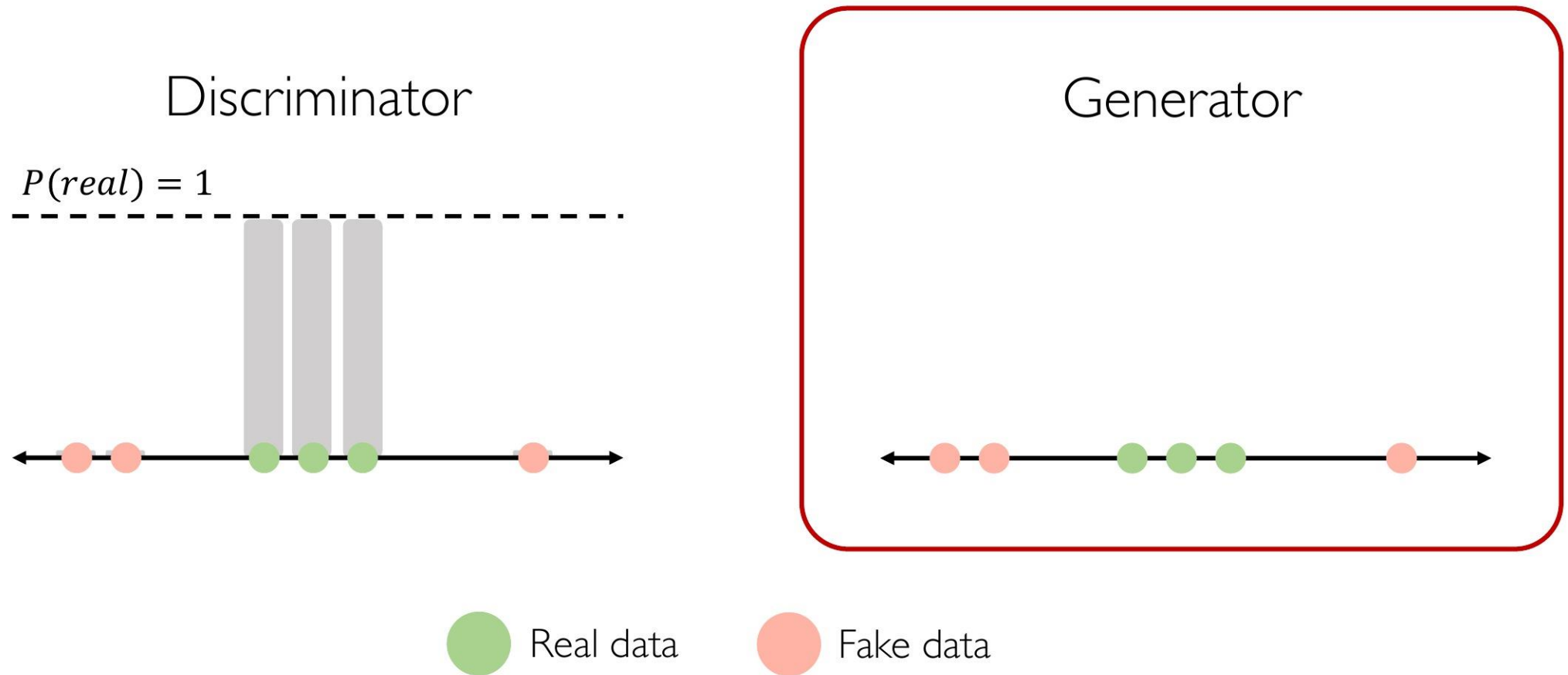


● Real data

● Fake data

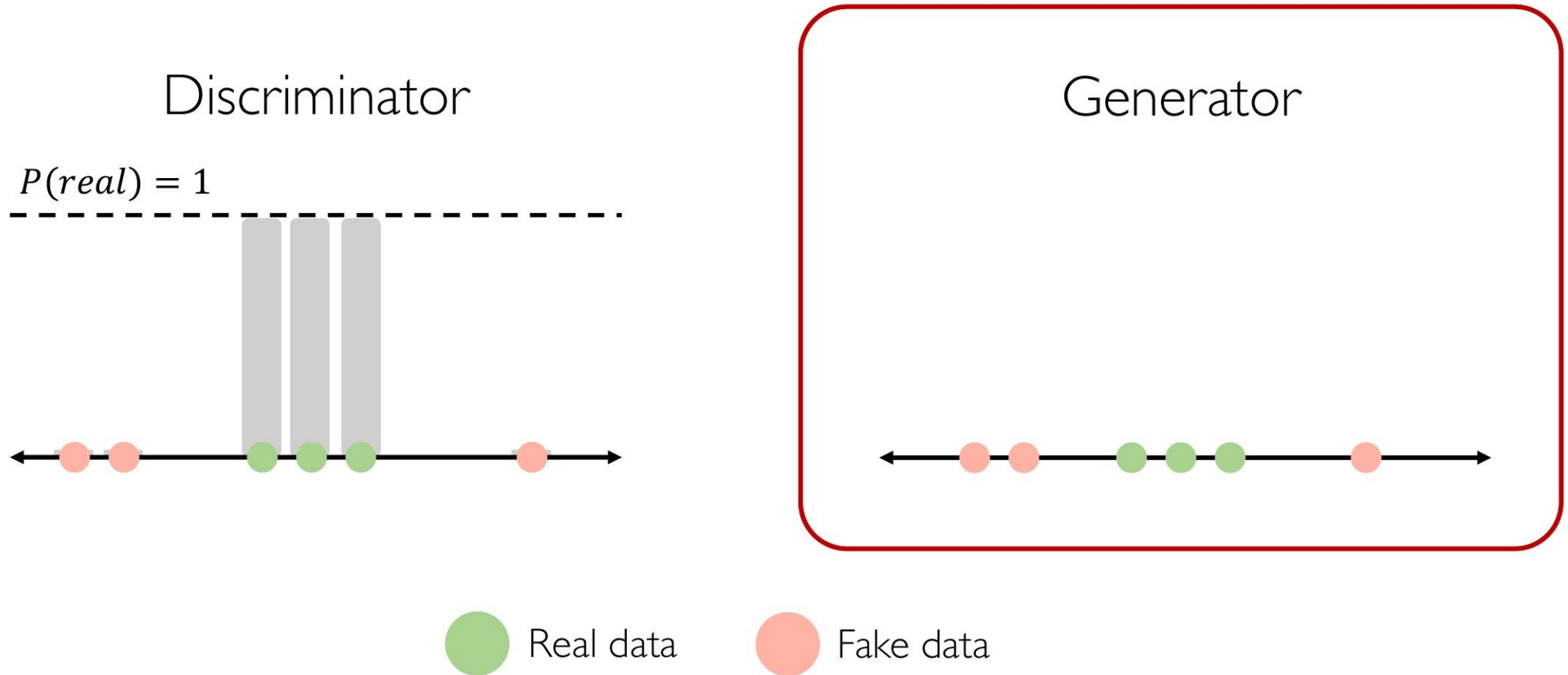
Intuition behind GANs

Generator tries to improve its imitation of the data.



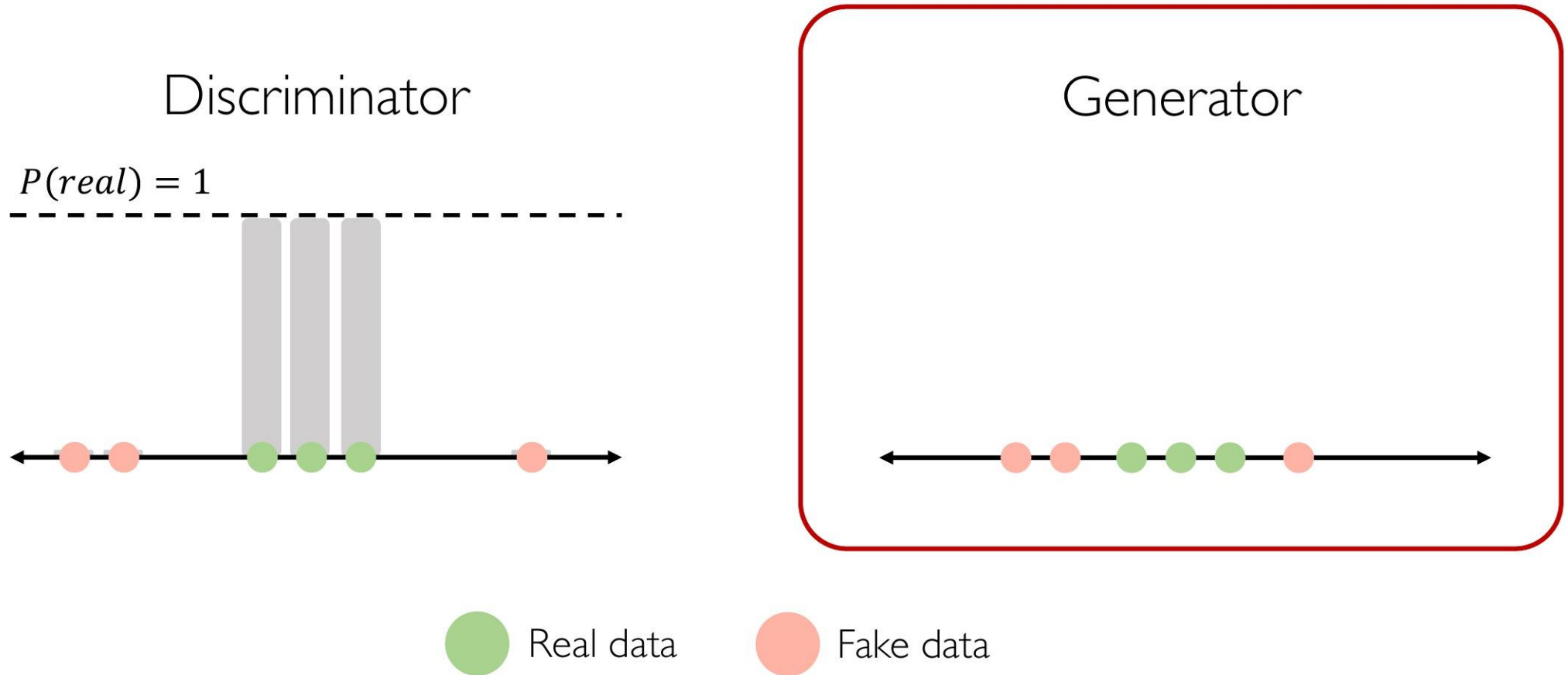
Intuition behind GANs

Generator tries to improve its imitation of the data.



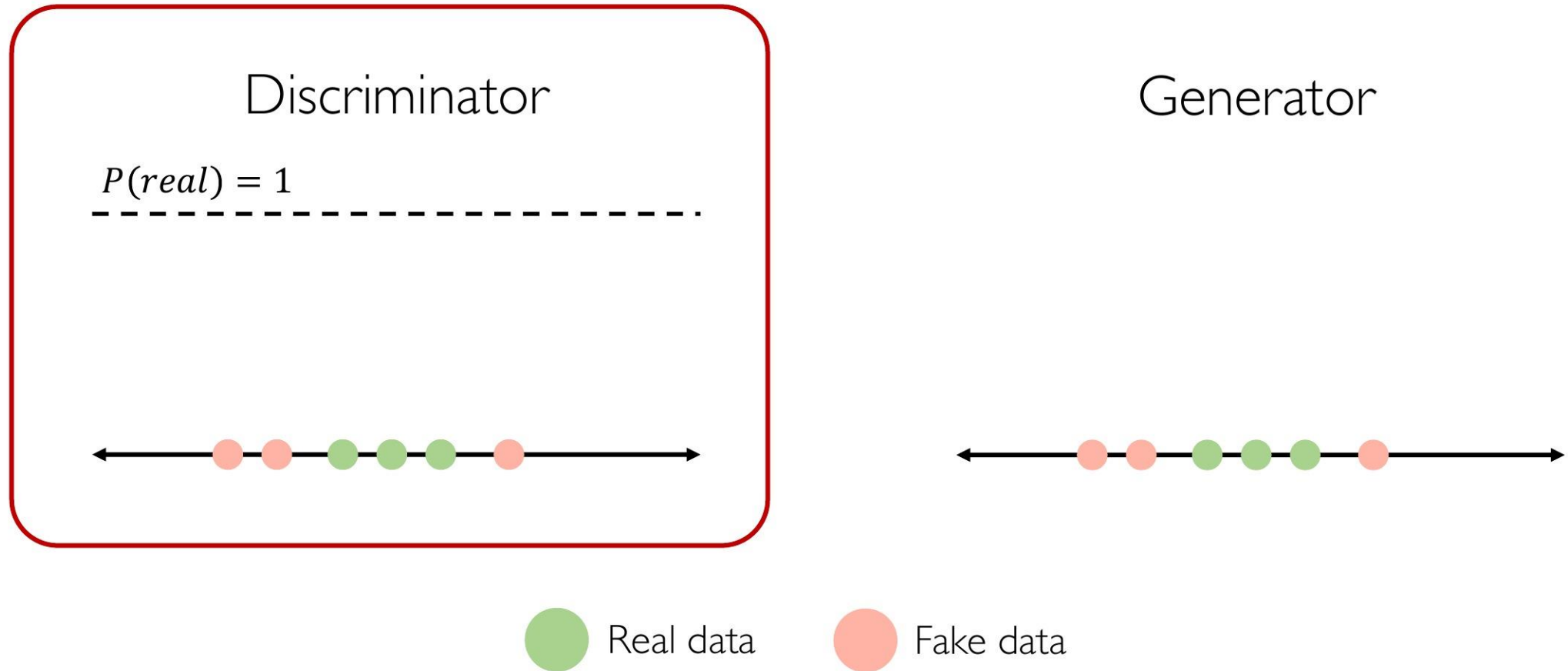
Intuition behind GANs

Generator tries to improve its imitation of the data.



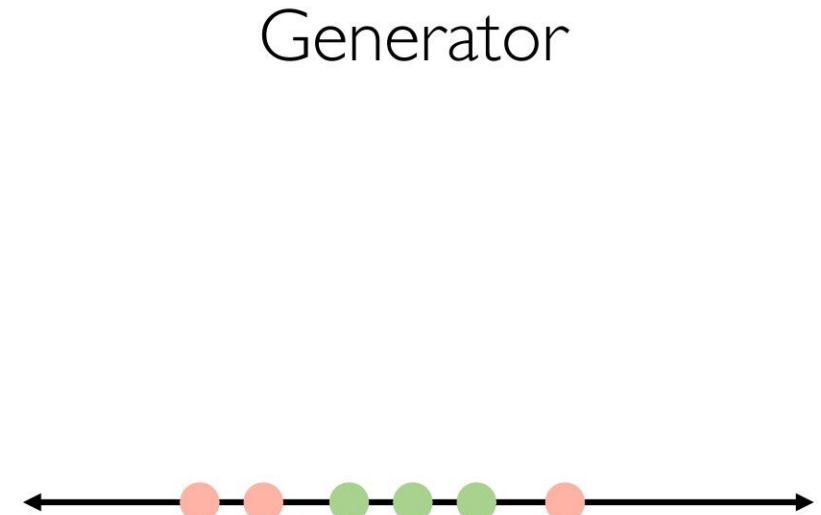
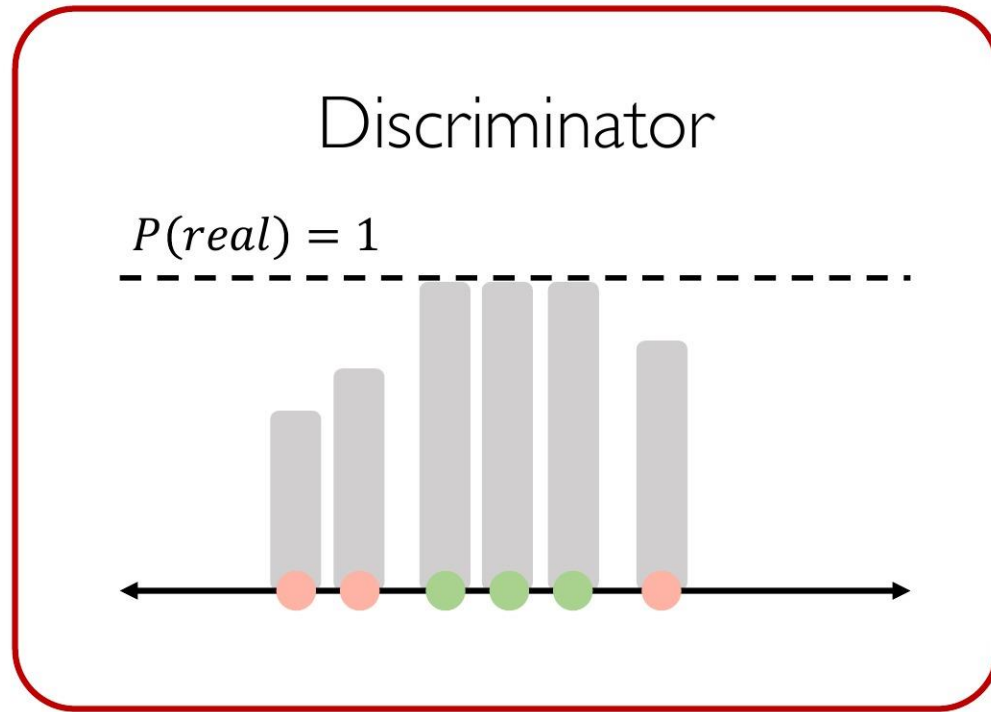
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

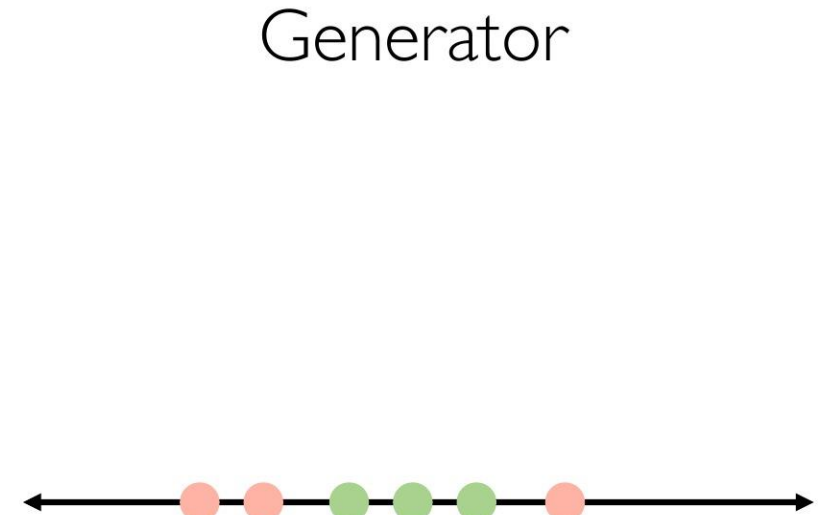
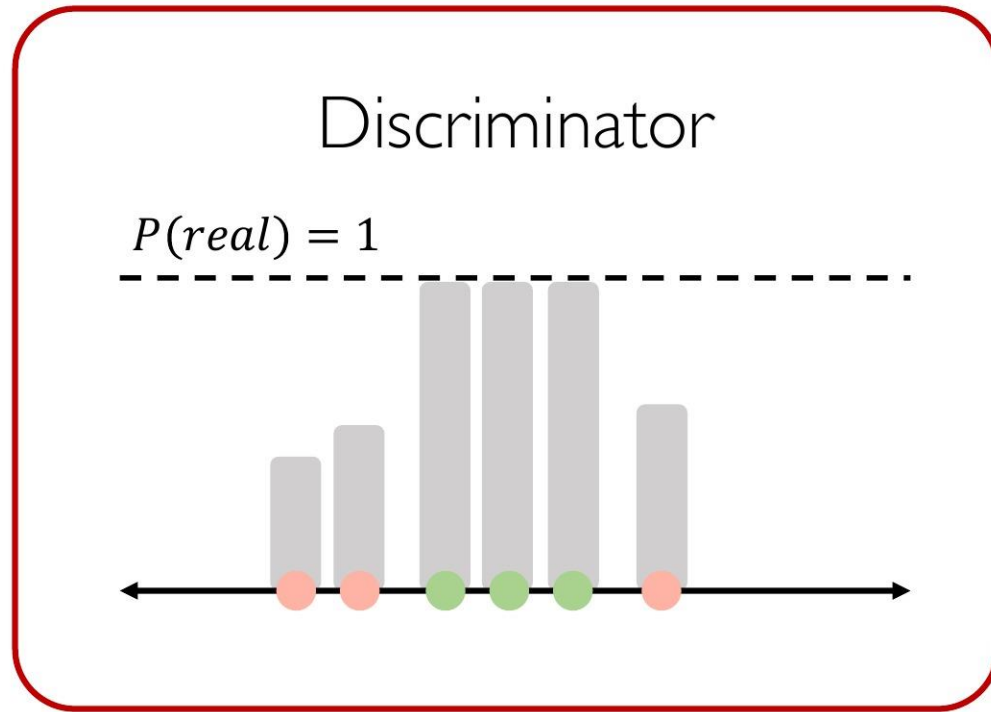


● Real data

● Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

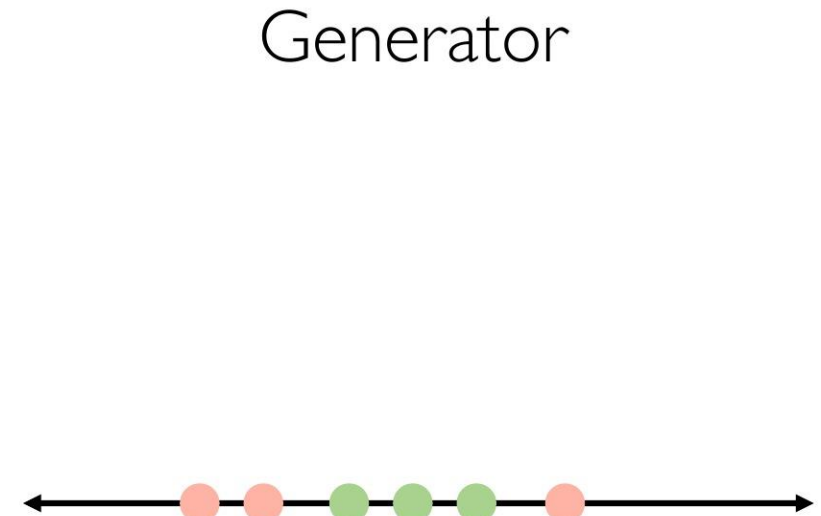
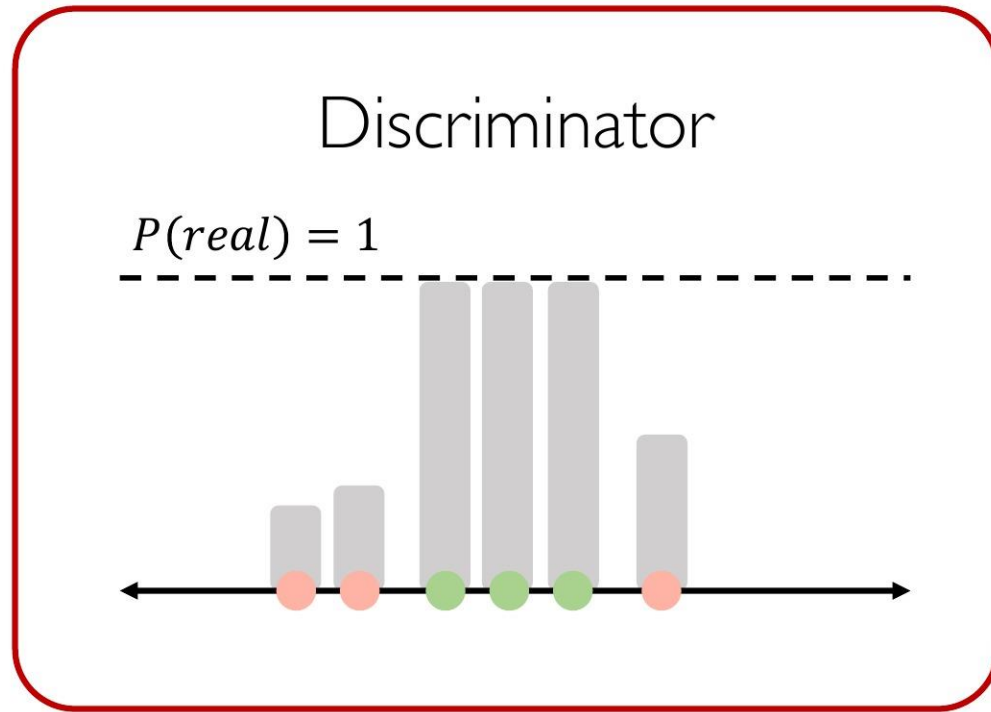


● Real data

● Fake data

Intuition behind GANs

Discriminator tries to predict what's real and what's fake.

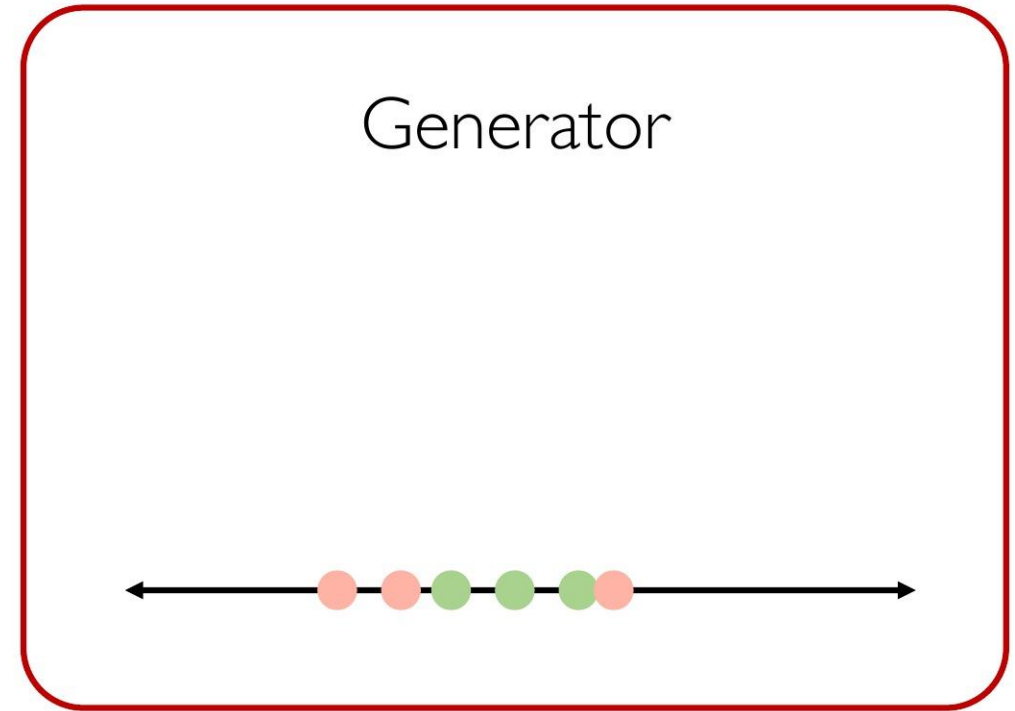
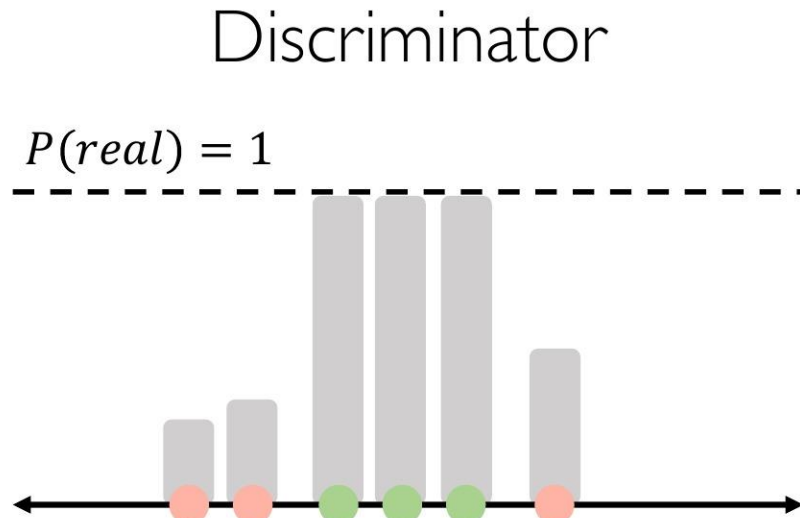


● Real data

● Fake data

Intuition behind GANs

Generator tries to improve its imitation of the data.

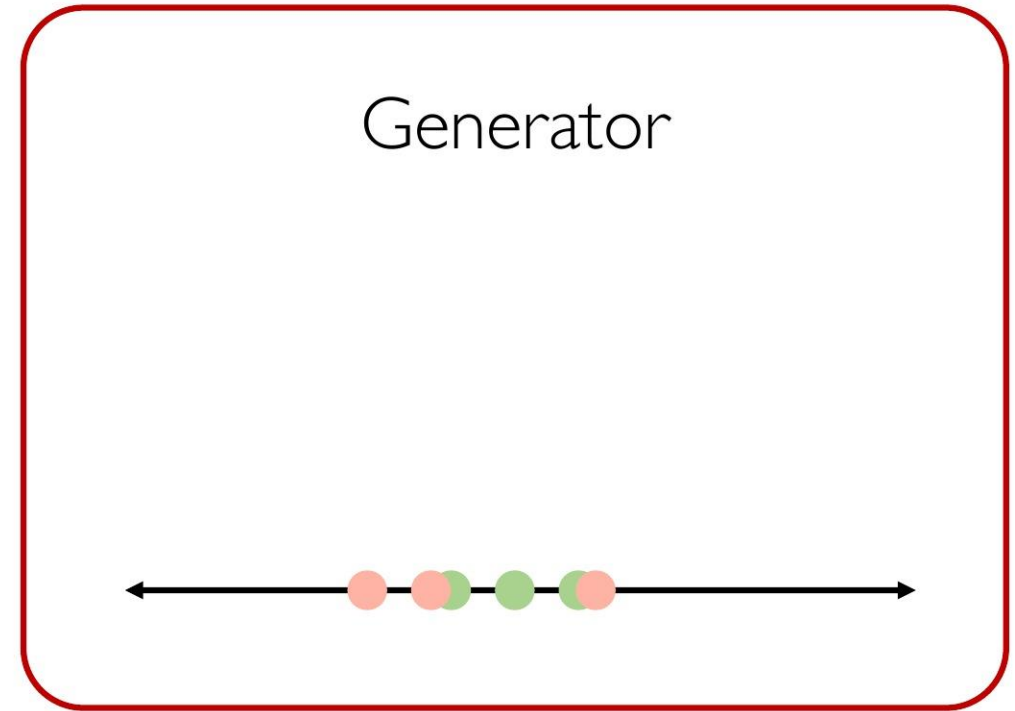
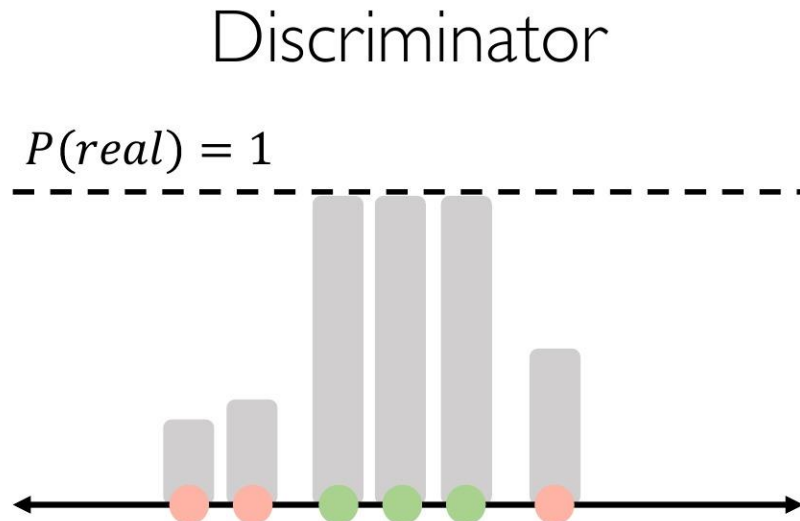


● Real data

● Fake data

Intuition behind GANs

Generator tries to improve its imitation of the data.

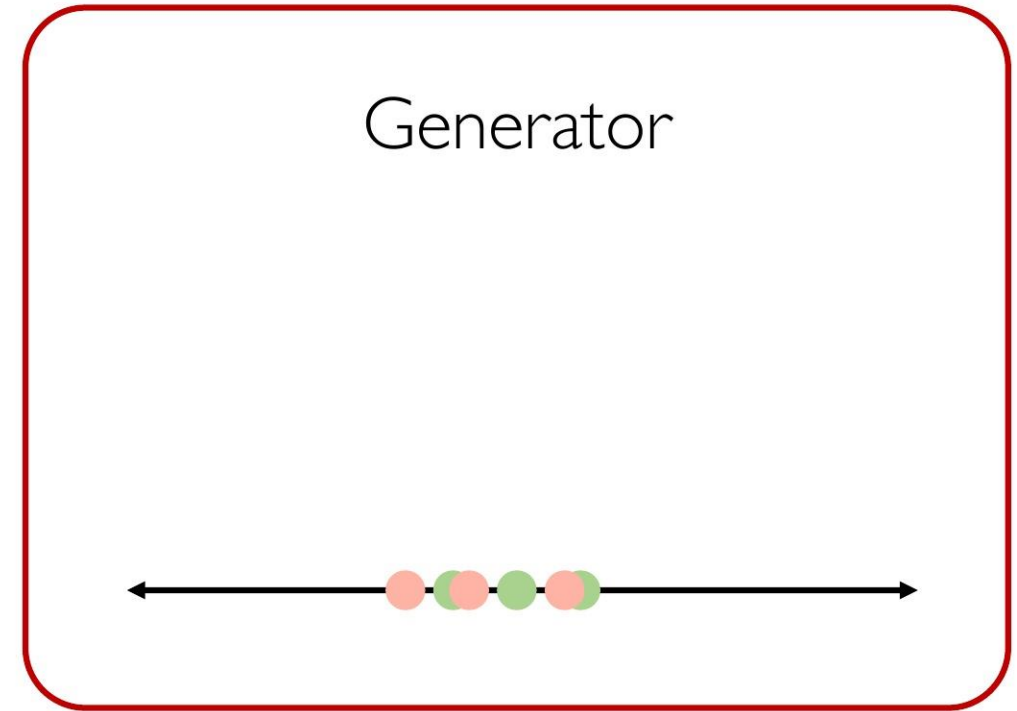
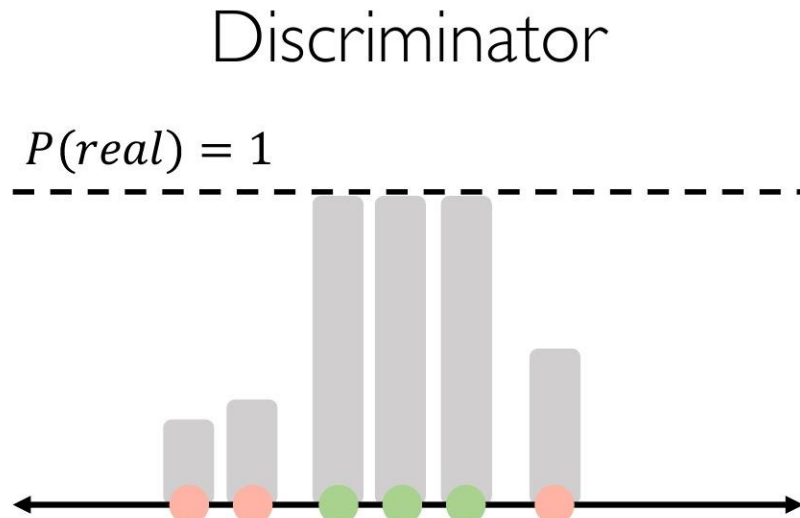


● Real data

● Fake data

Intuition behind GANs

Generator tries to improve its imitation of the data.



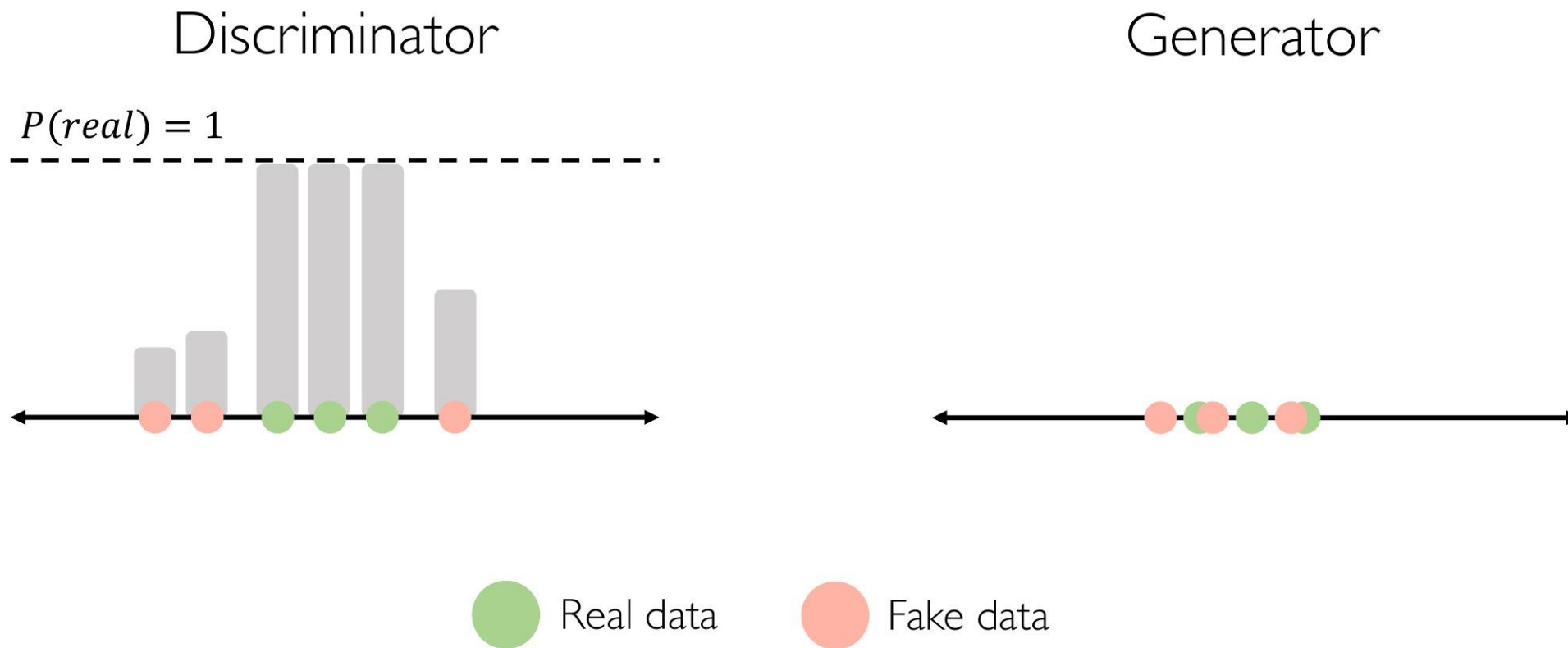
● Real data

● Fake data

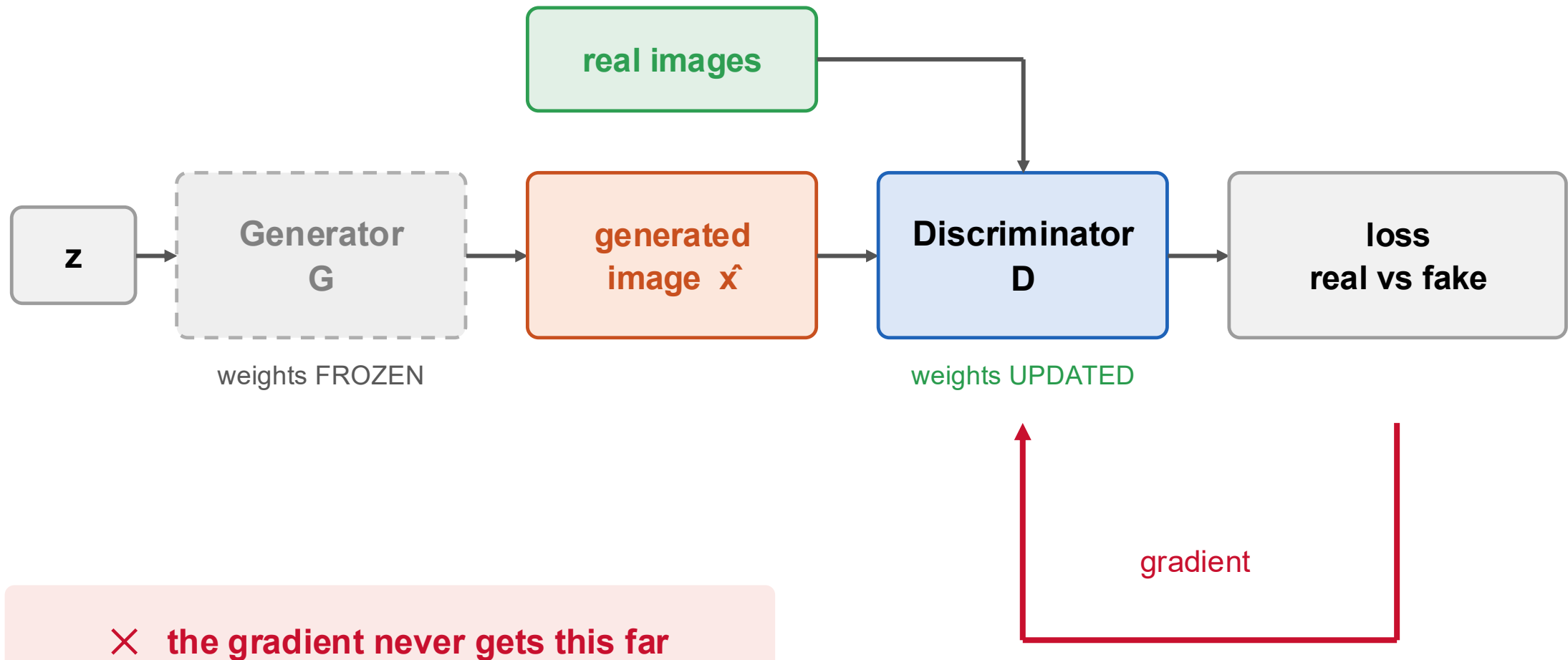
Intuition behind GANs

Discriminator tries to identify real data from fakes created by the generator.

Generator tries to create imitations of data to trick the discriminator.



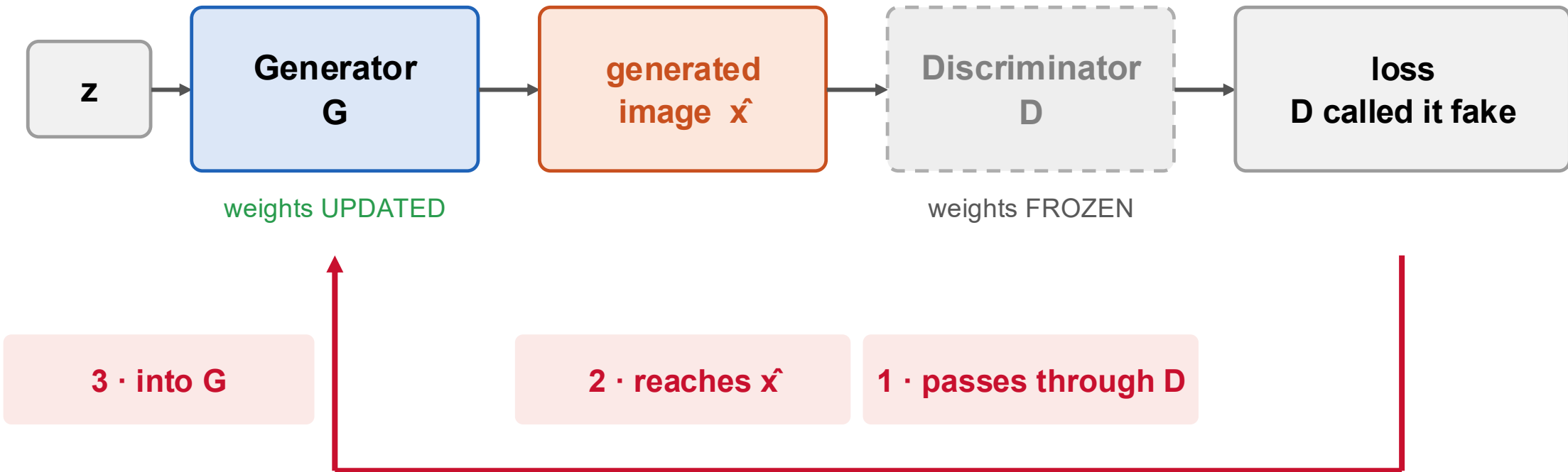
Step 1 — training the discriminator



Backprop stops at D. Only D's weights change; G is held fixed and produces the fakes as data.

Step 2 — training the generator

The forward pass is exactly the same as before — only the backward path changes.



The gradient must travel through D to reach G — but D 's weights are not updated. D is only a path.

G/D Game

Training procedure, we want to minimize:

Labels: $\begin{cases} y_{real} & \text{is always 1} \\ y_{gen} & \text{is always 0} \end{cases}$

- The cost of the discriminator

$$J^{(D)} = \underbrace{-\frac{1}{m_{real}} \sum_{i=1}^{m_{real}} y_{real}^{(i)} \cdot \log(D(x^{(i)}))}_{\text{cross-entropy 1: "D should correctly label real data as 1"}} - \underbrace{\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} (1 - y_{gen}^{(i)}) \cdot \log(1 - D(G(z^{(i)})))}_{\text{cross-entropy 2: "D should correctly label generated data as 0"}}$$

- The cost of the generator

$$J^{(G)} = -J^{(D)} = \frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)})))$$

“G should try to fool D: by minimizing the opposite of what D is trying to minimize”

Why we don't use $\log(1 - D(G(z)))$

The saturating generator loss, the problem with it, and the one-line fix

- $\log(1 - D(G(z)))$ — saturating
- $-\log D(G(z))$ — non-saturating

The generator's half of the game

THE SETUP

THE ORIGINAL MINIMAX OBJECTIVE

$$\min_G \max_D E[\log D(x)] \\ + E[\log(1 - D(G(z)))]$$

D trains on both terms.

G appears in the second one only — that is the whole lever it has.

What **G** is trying to do

- Fool the discriminator — push $D(G(z))$ up towards 1.
- It cannot touch the real-data term, so it minimises $\log(1 - D(G(z)))$.
- Perfectly sensible on paper: the loss falls as $D(G(z))$ rises.

So where is the catch? Look at where training actually starts.

Two generator losses, same goal

THE TWO CANDIDATES

ORIGINAL MINIMAX GENERATOR OBJECTIVE

$$\min_G \mathcal{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

BUT IN PRACTICE WE USE THE NON-SATURATING LOSS

$$L_G = - \mathcal{E}_{z \sim p_z} [\log D(G(z))]$$

Both are minimised when $D(G(z)) = 1$ — the same fixed point. Everything that follows is about the road between here and there.

First, rule out the obvious suspect

THE PROBLEM

- 1 Early in training the generator is bad**
Its samples are obviously fake, so the discriminator recognises them easily: $D(G(z)) \approx 0$.
- 2 Substitute that into the original loss**
 $L_G = \log(1 - D(G(z))) \approx \log(1) = 0$. The loss value is small — nothing alarming there.
- 3 So the loss itself is not the main problem**
The trouble only shows up when we differentiate it. That is what the next two slides do.

The key problem is vanishing gradients when the discriminator is very good.

Gradient of the original loss

DERIVATION 1 · SATURATING

Let $\mathbf{D}(\mathbf{G}(\mathbf{z})) = \sigma(a)$, where a is the discriminator's output *before* the sigmoid.

THE LOSS $L_G = \log(1 - \sigma(a))$

DIFFERENTIATE $\partial L_G / \partial a = -\sigma'(a) / (1 - \sigma(a))$

SIGMOID RULE $= -\sigma(a)(1 - \sigma(a)) / (1 - \sigma(a)) = -\sigma(a)$

$$\partial L_G / \partial a = -D(G(z))$$

If the discriminator is confident the sample is fake, $D(G(z)) \approx 0$, then $\partial L_G / \partial a \approx 0$.

The generator gets almost no gradient — and this is exactly the regime training starts in.

Gradient of the non-saturating loss

DERIVATION 2 · NON-SATURATING

Same substitution: $\mathbf{D}(\mathbf{G}(\mathbf{z})) = \sigma(\mathbf{a})$. Only the loss changes.

THE LOSS $L_G = -\log \sigma(\mathbf{a})$

DIFFERENTIATE $\partial L_G / \partial \mathbf{a} = -(1 / \sigma(\mathbf{a})) \cdot \sigma(\mathbf{a})(1 - \sigma(\mathbf{a}))$

SIMPLIFY $= -(1 - \sigma(\mathbf{a}))$

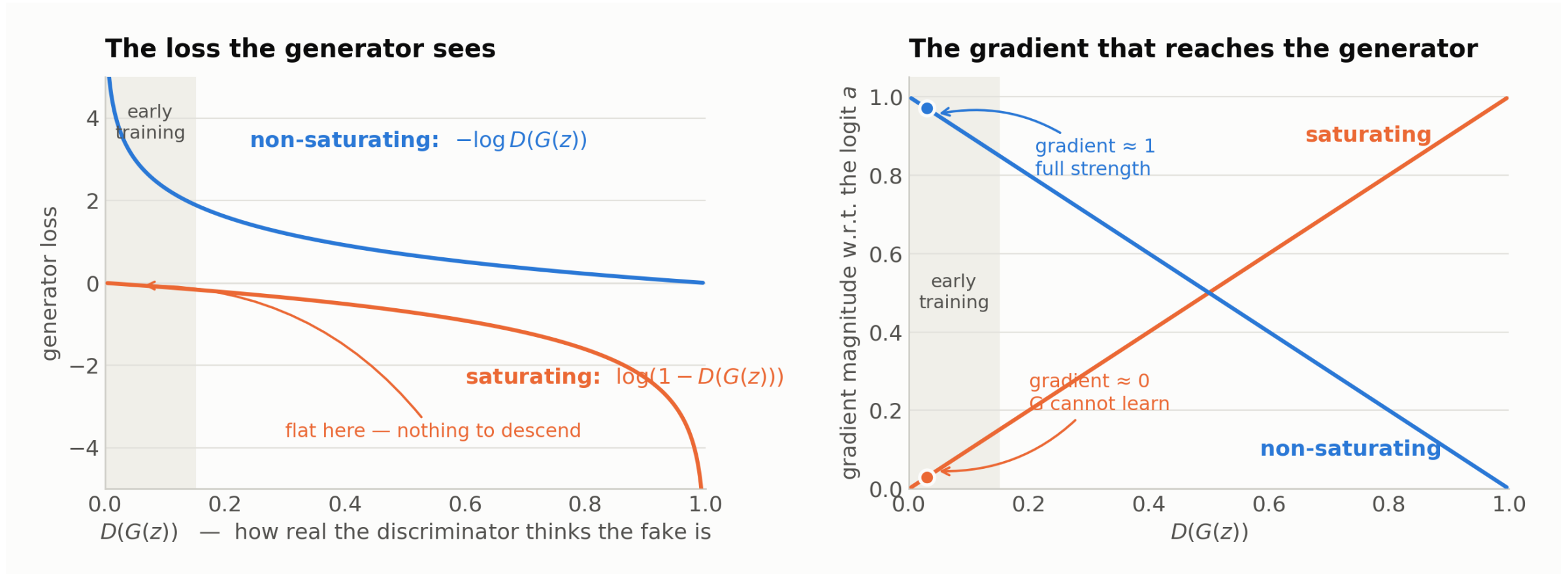
$$\partial L_G / \partial \mathbf{a} = \mathbf{D}(\mathbf{G}(\mathbf{z})) - 1$$

Now $\mathbf{D}(\mathbf{G}(\mathbf{z})) \approx 0$ gives $\partial L_G / \partial \mathbf{a} \approx -1$ — a strong gradient, saying:

“change your output so the discriminator becomes more likely to call it real.”

Two losses, two very different gradients

THE PICTURE



Left: the orange loss is flat exactly where training begins. **Right:** its gradient is $D(G(z))$ itself — ≈ 0 there, while the blue one is at full strength.

The comparison that matters

SIDE BY SIDE

Discriminator output	Original $\log(1 - D(G(z)))$	Non-saturating $-\log D(G(z))$
$D(G(z)) \approx 0$ (early training)	gradient ≈ 0 no learning signal ✗	gradient ≈ -1 strong learning signal ✓
$D(G(z)) \approx 1$ (generator winning)	gradient ≈ -1	gradient ≈ 0

Early in GAN training $D(G(z)) \approx 0$, because the generated images are obviously fake — so the top row is the row that matters.

The original minimax loss lets the generator's gradients saturate exactly there, making learning extremely slow.

In your code, and in your training logs

IN PRACTICE

```
# saturating — don't
loss_G = torch.log(1 - D(G(z))).mean()

# non-saturating — do
loss_G = bce(D(G(z)), ones_like(...))
```

Label the fakes as real in the generator step — that is all "non-saturating" means in code.

It fixes the vanishing gradient. It does not fix mode collapse — that is a different problem, and it is still waiting for you.

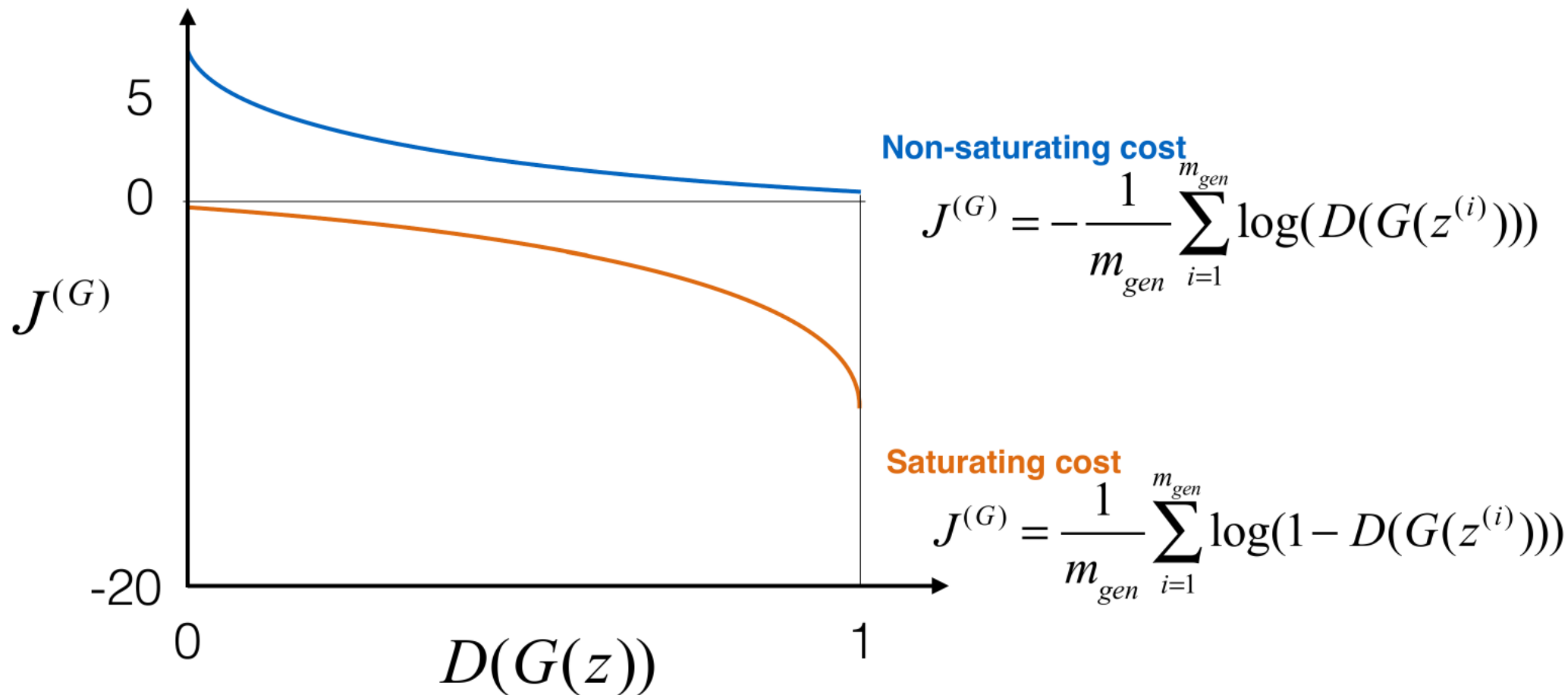
Symptoms of getting it wrong

- Generator loss pinned near zero and barely moving.
- Discriminator loss collapsing towards zero within a few epochs.
- Sample grids that stop changing while the loss curves look "fine".

Training GANs

Saturating cost
for the generator:

$$\min \left[\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)}))) \right] \Leftrightarrow \max \left[\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(D(G(z^{(i)}))) \right] \Leftrightarrow \min \left[-\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(D(G(z^{(i)}))) \right]$$



Training GANs

Note that:

$$\min \left[\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)}))) \right] \Leftrightarrow \max \left[\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(D(G(z^{(i)}))) \right] \Leftrightarrow \min \left[-\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(D(G(z^{(i)}))) \right]$$

New training procedure, we want to minimize:

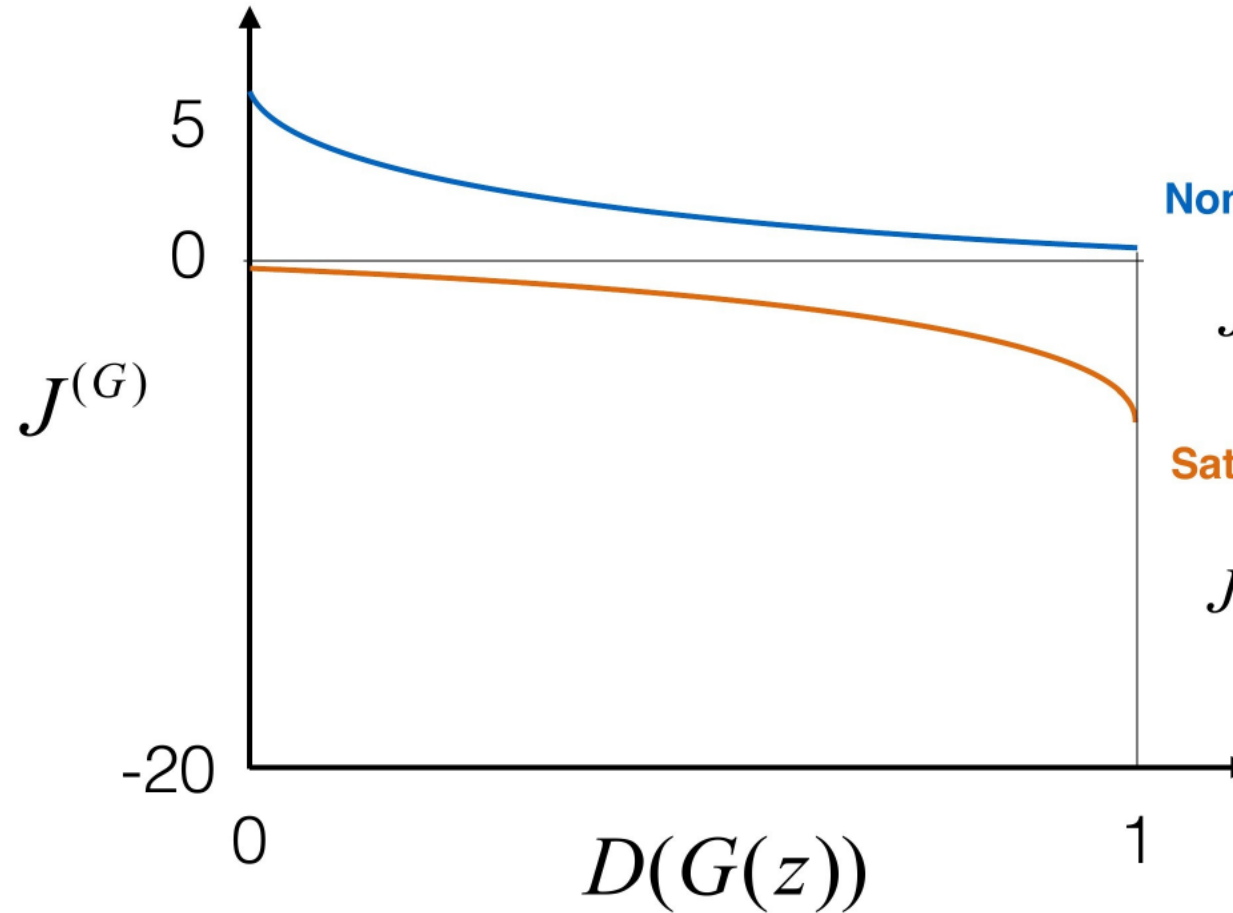
$$J^{(D)} = \underbrace{-\frac{1}{m_{real}} \sum_{i=1}^{m_{real}} y_{real}^{(i)} \cdot \log(D(x^{(i)}))}_{\text{cross-entropy 1: "D should correctly label real data as 1"}} - \underbrace{\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} (1 - y_{gen}^{(i)}) \cdot \log(1 - D(G(z^{(i)})))}_{\text{cross-entropy 2: "D should correctly label generated data as 0"}}$$

$$J^{(G)} = -\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(D(G(z^{(i)}))) \quad \text{"G should try to fool D: by minimizing this"}$$

Training GANs

Simultaneously training G/D?

```
for num_iterations:  
    for k iterations:  
        update D  
    update G
```



Non-saturating cost

$$J^{(G)} = -\frac{1}{m_g} \sum_{i=1}^{m_g} \log(D(G(z^{(i)})))$$

Saturating cost

$$J^{(G)} = \frac{1}{m_g} \sum_{i=1}^{m_g} \log(1 - D(G(z^{(i)})))$$

Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Putting it together: GAN training algorithm

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

Training GANs: Two-player game

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Putting it together: GAN training algorithm

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by ascending its stochastic gradient (improved objective):

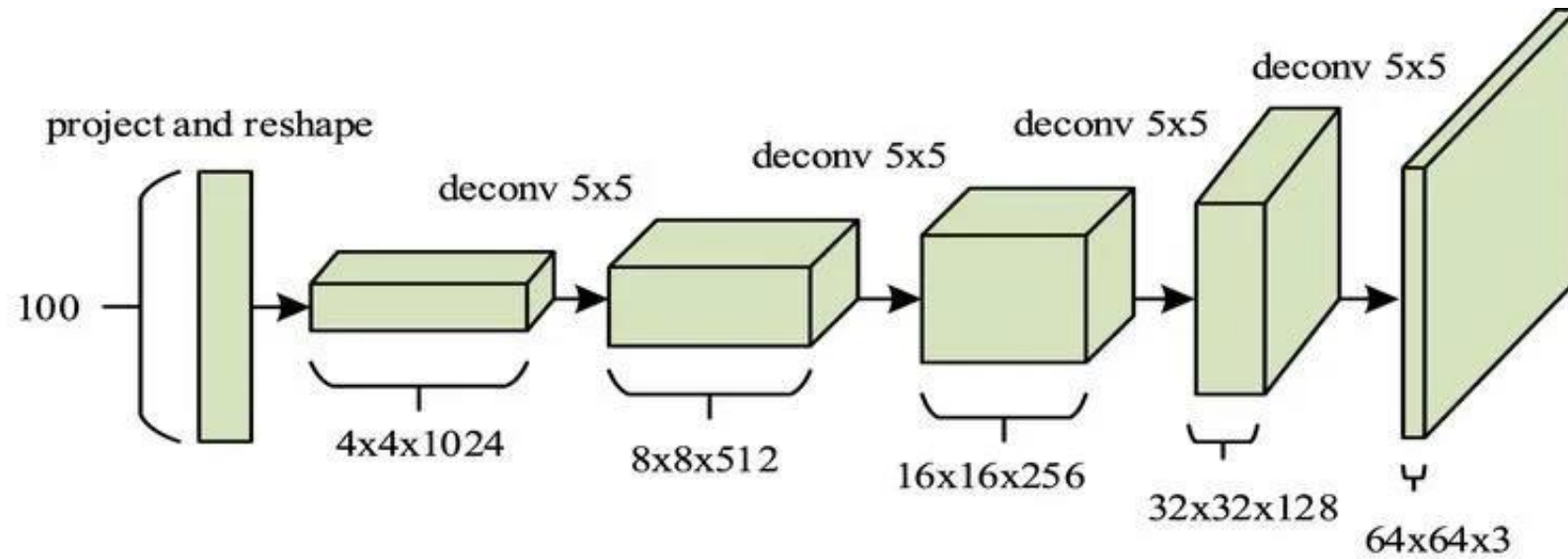
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

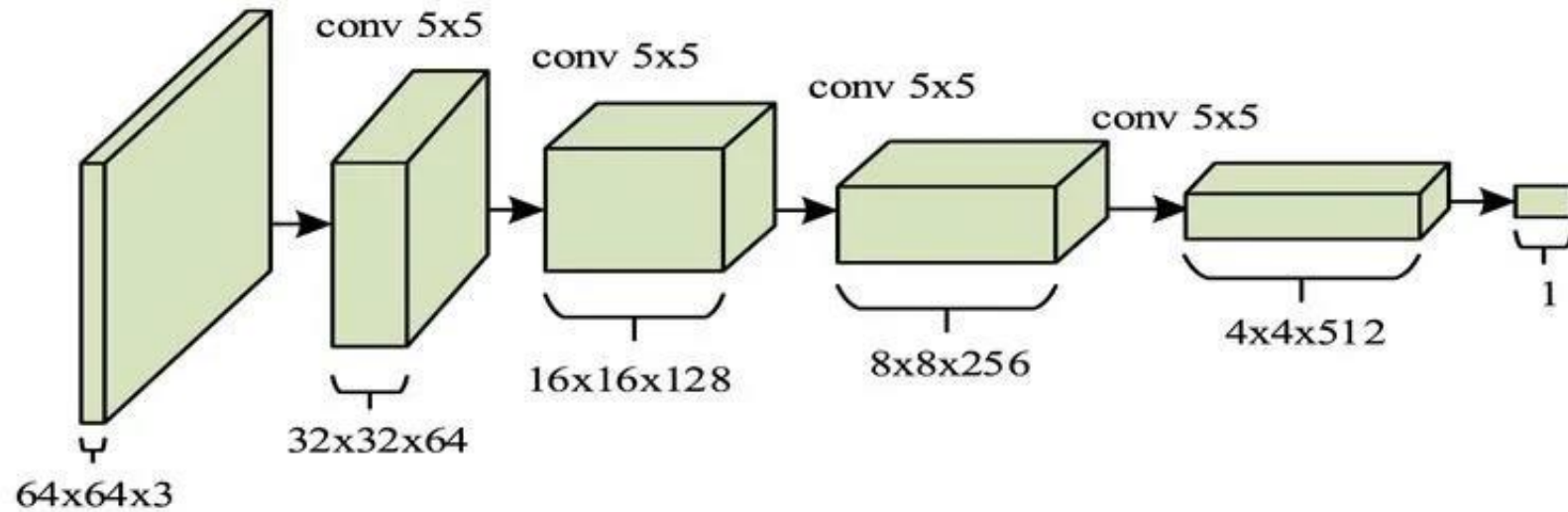
Some find $k=1$
more stable,
others use $k > 1$,
no best rule.

Recent work (e.g.
Wasserstein GAN)
alleviates this
problem, better
stability!

Generative Adversarial Nets: Convolutional Architectures



(a) Generator



(b) Discriminator

GAN Challenges

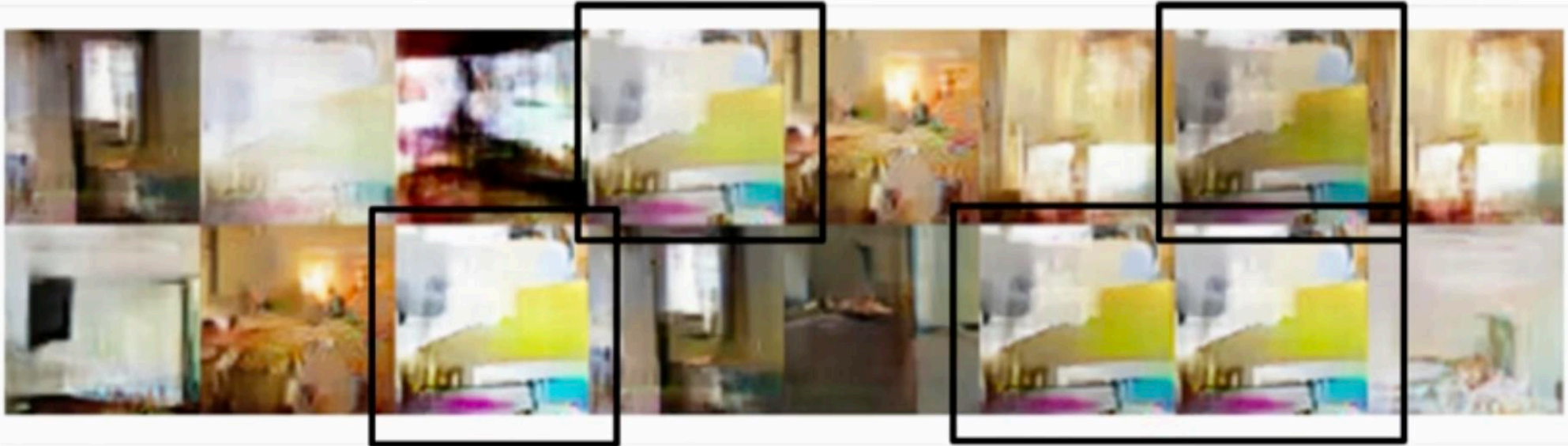
- Vanishing Gradient in Generators
- Oscillating loss
- Uninformative loss
- Mode collapse
- Hyperparameter Sensitivity

Vanishing Gradient issue

- Discriminator is trained first
- Generator is lousy in the beginning
- Backpropagate for generator
- The discriminator has already been trained
- The solution is non-saturating loss already discussed in previous slides

Mode Collapse

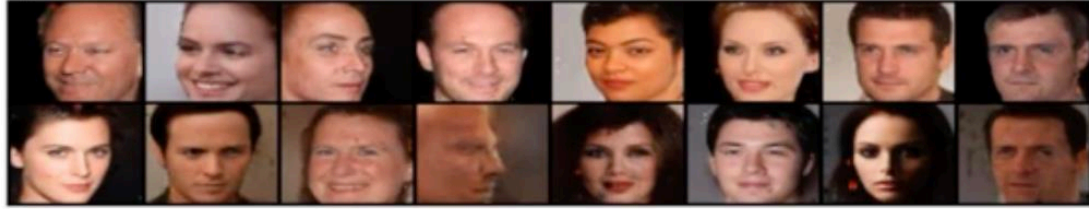
- GANs are notorious for suffering from **mode collapse**
- Intuitively, this refers to the phenomena where the generator of a GAN collapses to one or few samples (dubbed as “modes”)



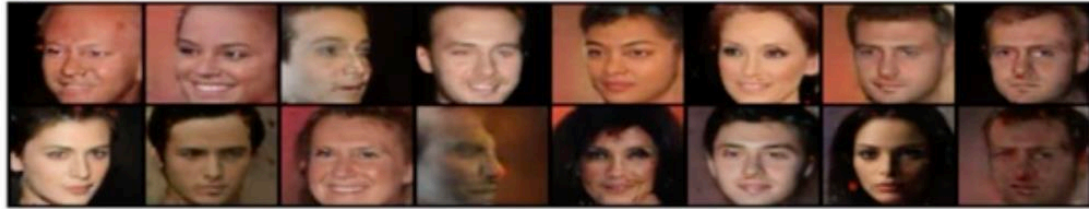
Arjovsky et al., 2017

Mode Collapse

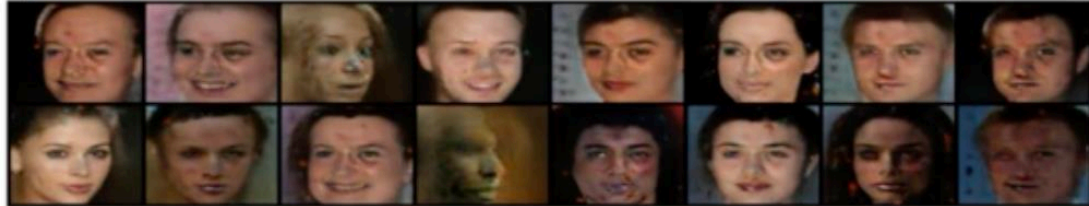
epoch 350k



epoch 390k



epoch 396k



During the training, the generator may collapse to a setting where it always produces same output. This is called as MODE COLLAPSE

epoch 400k

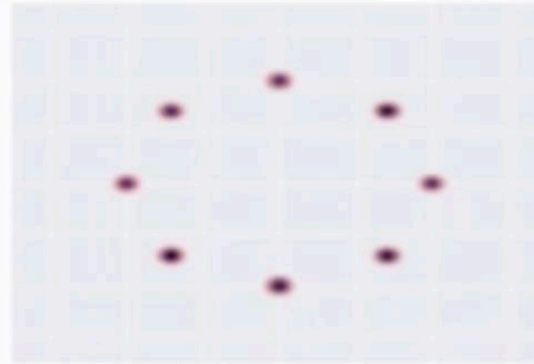


epoch 480k



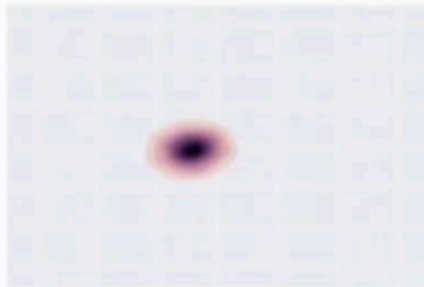
Mode collapse happens when the **generator fails to capture the full diversity of the real data distribution** and instead produces **only a few types (or even one type) of samples repeatedly**.

Mode Collapse

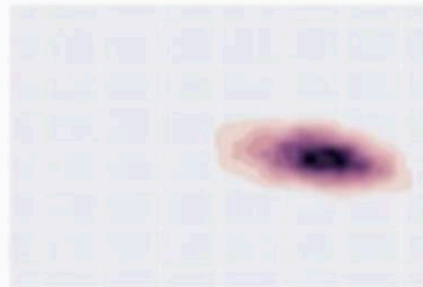


Target

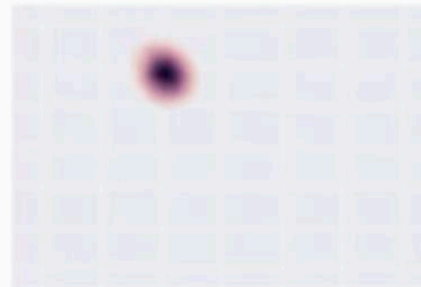
- True distribution is a mixture of Gaussians



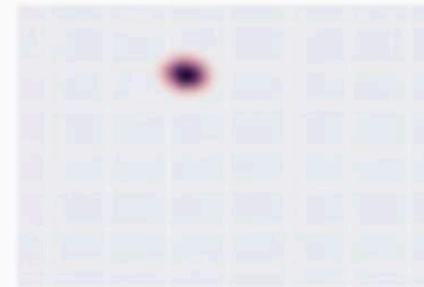
Step 0



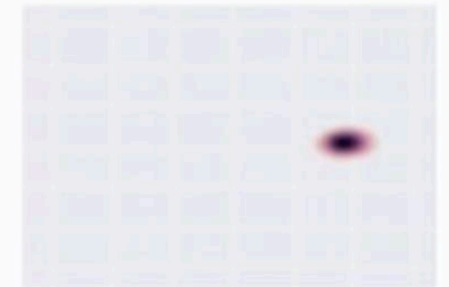
Step 5k



Step 10k



Step 15k

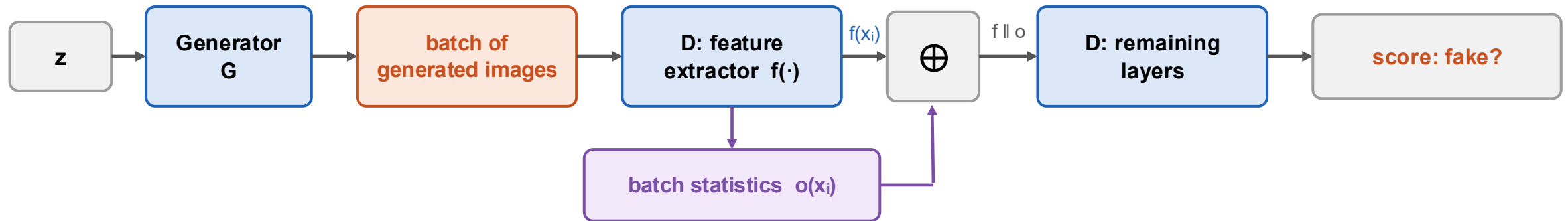


Step 20k

Solution: Minibatch discrimination: letting D see the whole batch

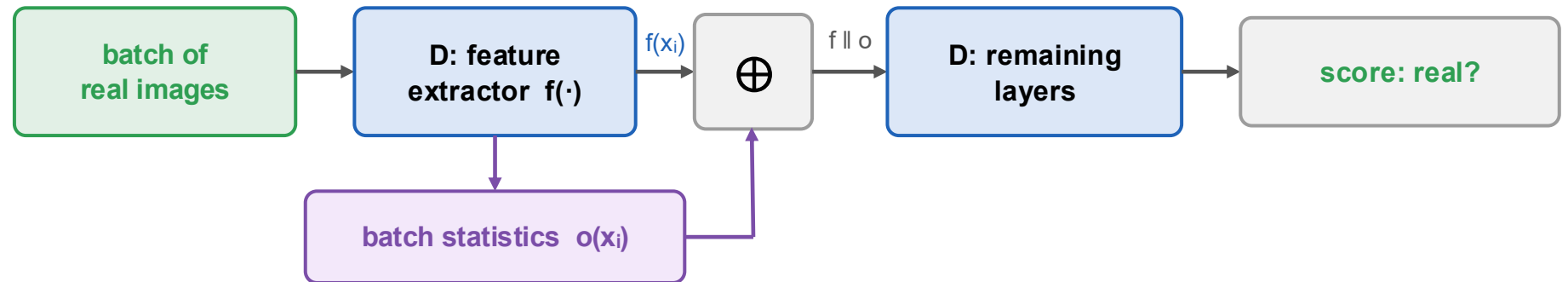
Mode collapse: G maps many different z to nearly the same image. A discriminator that scores one image at a time cannot see this — each sample on its own looks perfectly real.

GENERATED BATCH



$\updownarrow$ the same D — same weights, same $o(\cdot)$ module, both batches

REAL BATCH



$o(x_i)$ = how similar x_i is to the rest of its batch
mean / variance across the batch — or the pairwise form $\sum_j \exp(-\|M_i - M_j\|_1)$

Why it kills mode collapse: a collapsed batch has near-identical statistics, a real batch does not. D can now spot collapse directly, so G is punished for it.

Solution: Mini-Batch Discrimination

Instead of the discriminator looking at each sample independently, we add a special layer that compares samples **within the same batch** and gives the discriminator information about how diverse the batch is.

How it works:

Inside the discriminator, take features $f(x)$ of each image just before the final layer.

Compute a “similarity matrix” among samples in the batch (e.g., pairwise distances in feature space).

Append these diversity statistics as extra features into the discriminator’s last layer.

If all generated samples are very similar $\rightarrow$ low diversity signal $\rightarrow$ discriminator learns “these look fake.”

Penalty for generator: since the generator is trained to fool this stronger discriminator, it will get higher loss if it produces a low-diversity batch. To minimize loss, it must produce more diverse samples.

So “penalize the generator” here means: the generator is indirectly punished because the discriminator gets a stronger signal when the batch is mode-collapsed. There’s no explicit “diversity loss” added — the adversarial loss already handles it, but now with diversity-aware discriminator features.

Solution: Mini-Batch Discrimination

Normally:

You pass in **one image at a time**.

It extracts features (say, a 128-dim vector) → final layer → real/fake score.

Each image is judged **independently**.

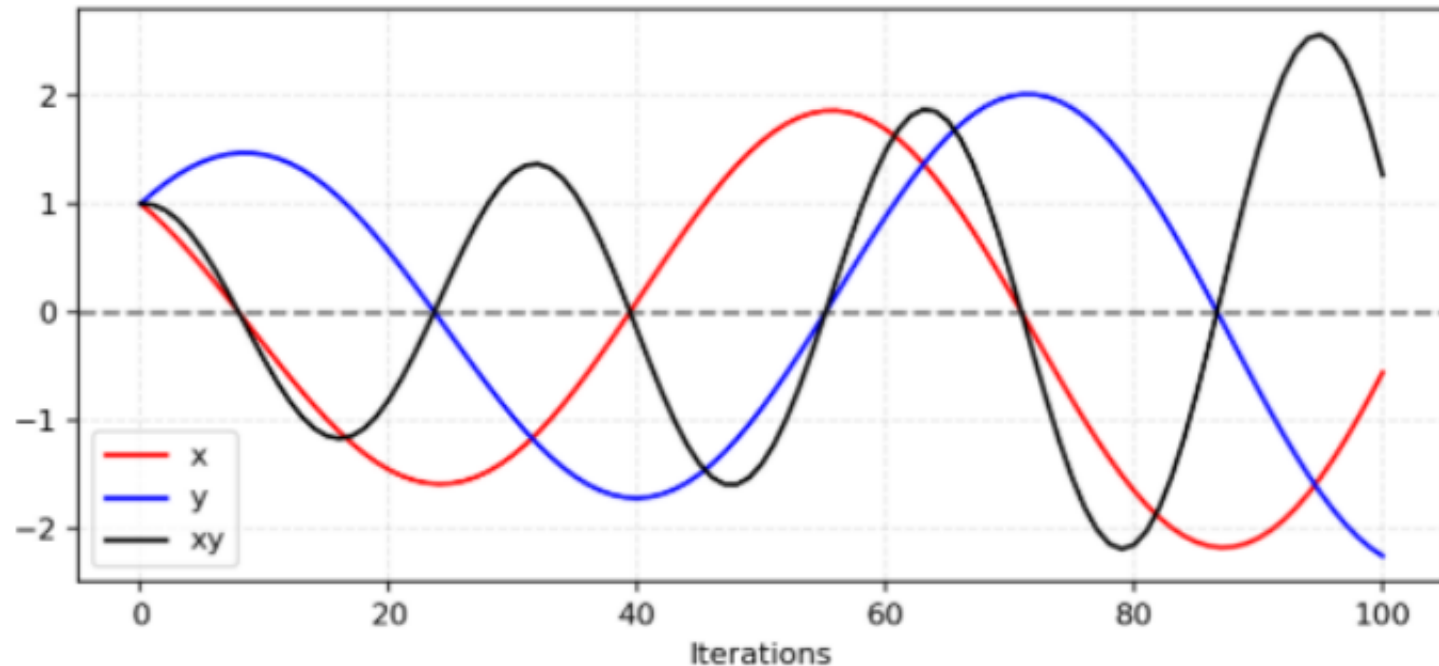
That's why mode collapse can slip through: If the generator outputs 10 copies of the same cat, The discriminator checks each copy alone → “looks real enough” → fooled.

Instead say:
“don't just look at one image in isolation — compare it with the other images in the same batch.”

1. **Extract features:** For each image, get its feature vector from the second-last layer (say `[0.2, -0.1, 0.5, ...]`).
2. **Compare samples:** For each image, measure *how similar its features are* to the features of the other images in the batch.
 - If 5 images are almost identical → their similarity scores will be very high.
 - If images are diverse → scores will be spread out.
3. **Add this info:** Take that “similarity score” and just **append it** to the feature vector before the last layer.
 - So instead of `[0.2, -0.1, 0.5, ...]`
 - You now have `[0.2, -0.1, 0.5, ..., similarity_score]`.
4. **Final judgment:** The discriminator uses this richer feature vector to decide real/fake.

Oscillating Loss

- **Oscillating loss** refers to the situation where the loss functions of the generator and discriminator fluctuate erratically during training, instead of converging smoothly to an optimal solution.
- The losses oscillate (i.e., go up and down), which makes it difficult to monitor the training process.
- **Why it's a problem:**
 - If the losses are oscillating, it often means that the GAN is not stabilizing its learning process.
 - This can be a sign that the discriminator and generator are not reaching a good equilibrium (nash equilibrium), resulting in inefficient training and slow convergence.



Oscillating Loss

- Potential solutions:**

- Adjust the learning rates:** A too-high learning rate for either the generator or the discriminator can cause the model to oscillate. Lowering the learning rate or using learning rate schedules can help.

- Use of batch normalization:** Normalizing the intermediate layers of the generator and discriminator helps stabilize training by ensuring the activations stay within a certain range.

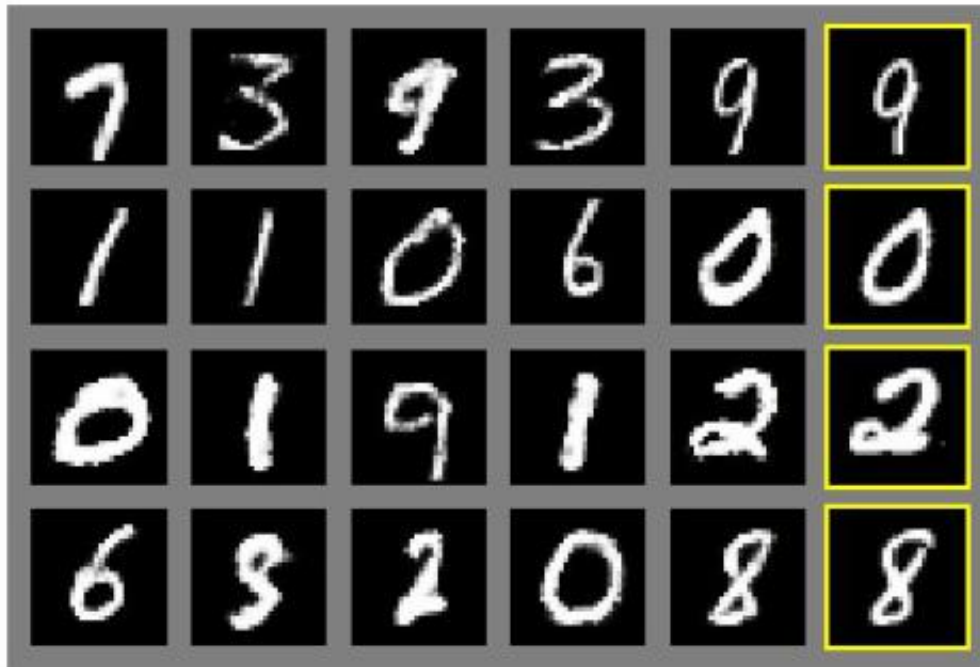
- Two-time scale update rule (TTUR):** This technique involves using different learning rates for the generator and discriminator, typically with a lower learning rate for the generator to ensure it doesn't overpower the discriminator.

Hyperparameter Sensitivity

- GANs are highly sensitive to hyperparameters, including learning rates, batch size, the number of layers, and the choice of activation functions.
- Even small changes in these hyperparameters can lead to drastically different results, making the training process difficult to control and tune.
- **Why it's a problem:** Since GANs can be sensitive to a wide range of hyperparameters, finding the right configuration for stable and effective training is often time-consuming and trial-and-error-based. This makes deploying GANs in real-world applications challenging

Generative Adversarial Nets

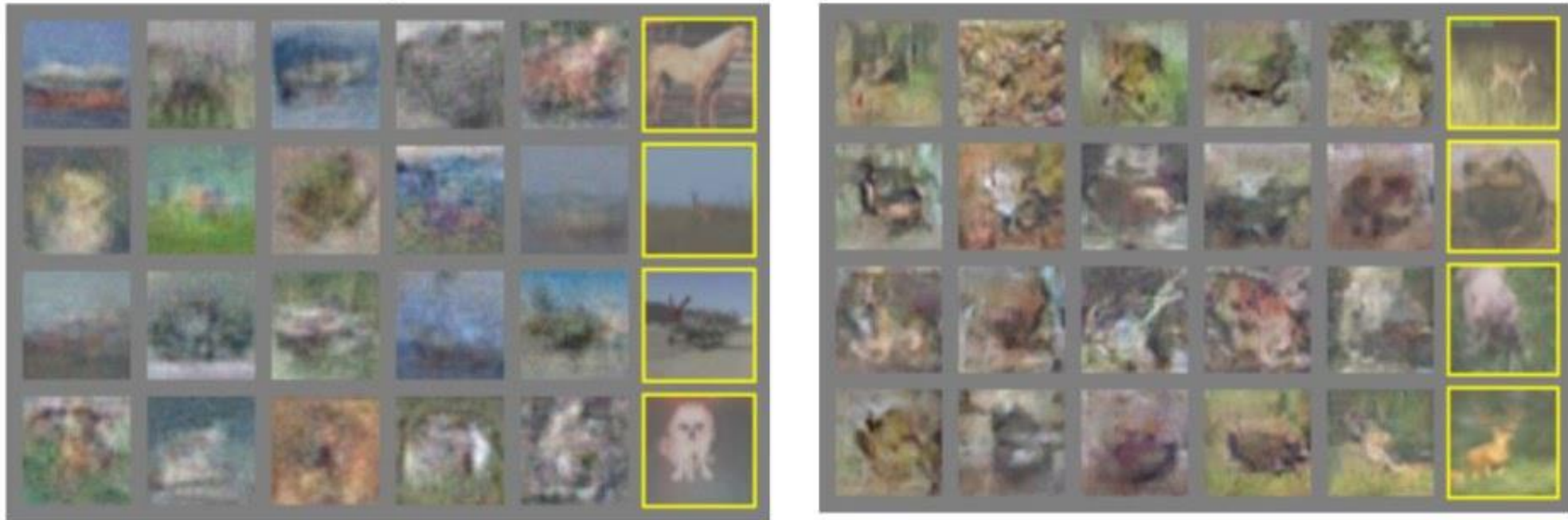
Generated samples



Nearest neighbor from training set

Generative Adversarial Nets

Generated samples (CIFAR-10)



Nearest neighbor from training set

Figures copyright Ian Goodfellow et al., 2014. Reproduced with permission.

Generative Adversarial Nets: Convolutional Architectures

Interpolating
between
random
points in latent
space



Radford et al,
ICLR 2016

Generative Adversarial Nets: Interpretable Vector Math

Radford et al, ICLR 2016

Smiling woman

Neutral woman

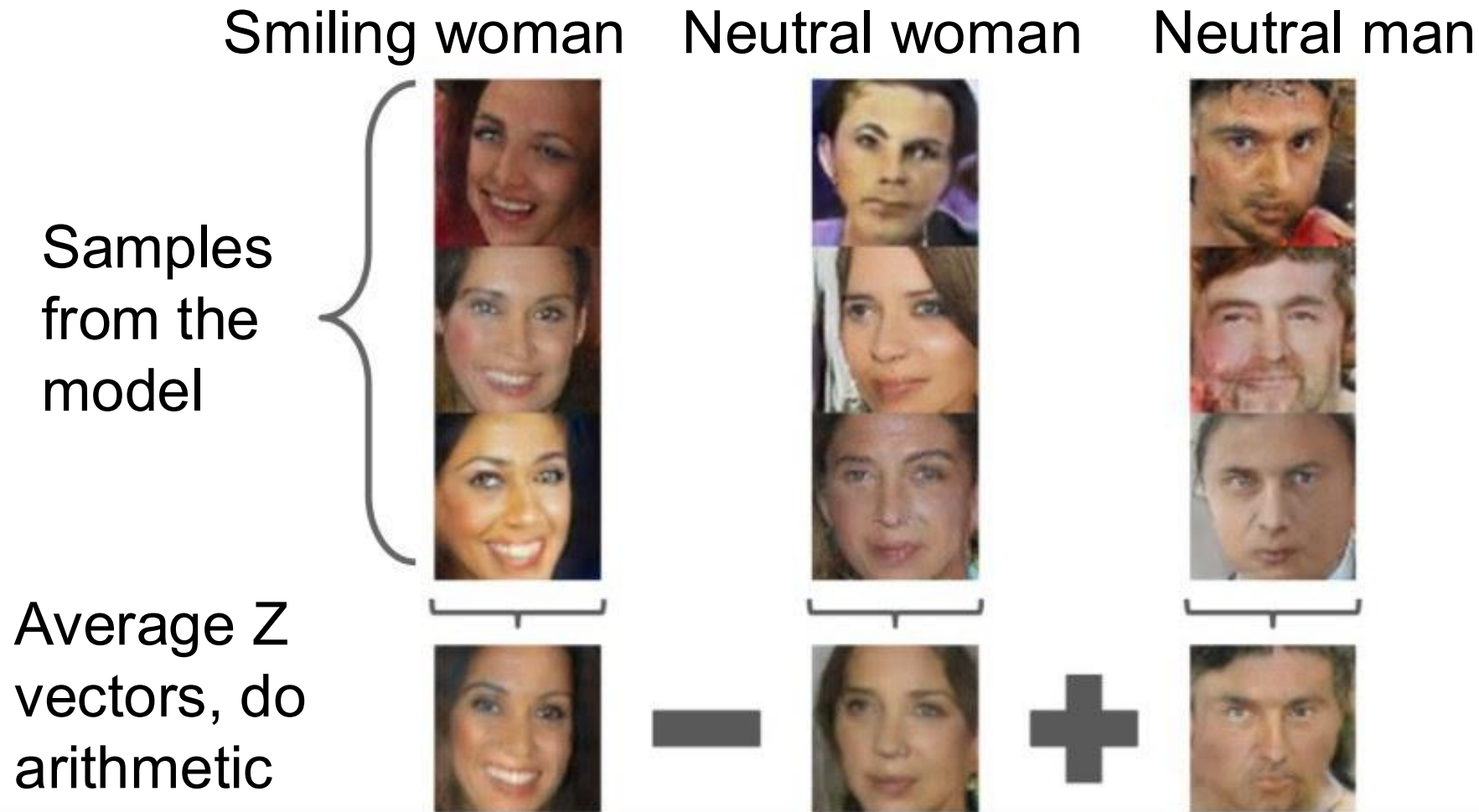
Neutral man

Samples
from the
model



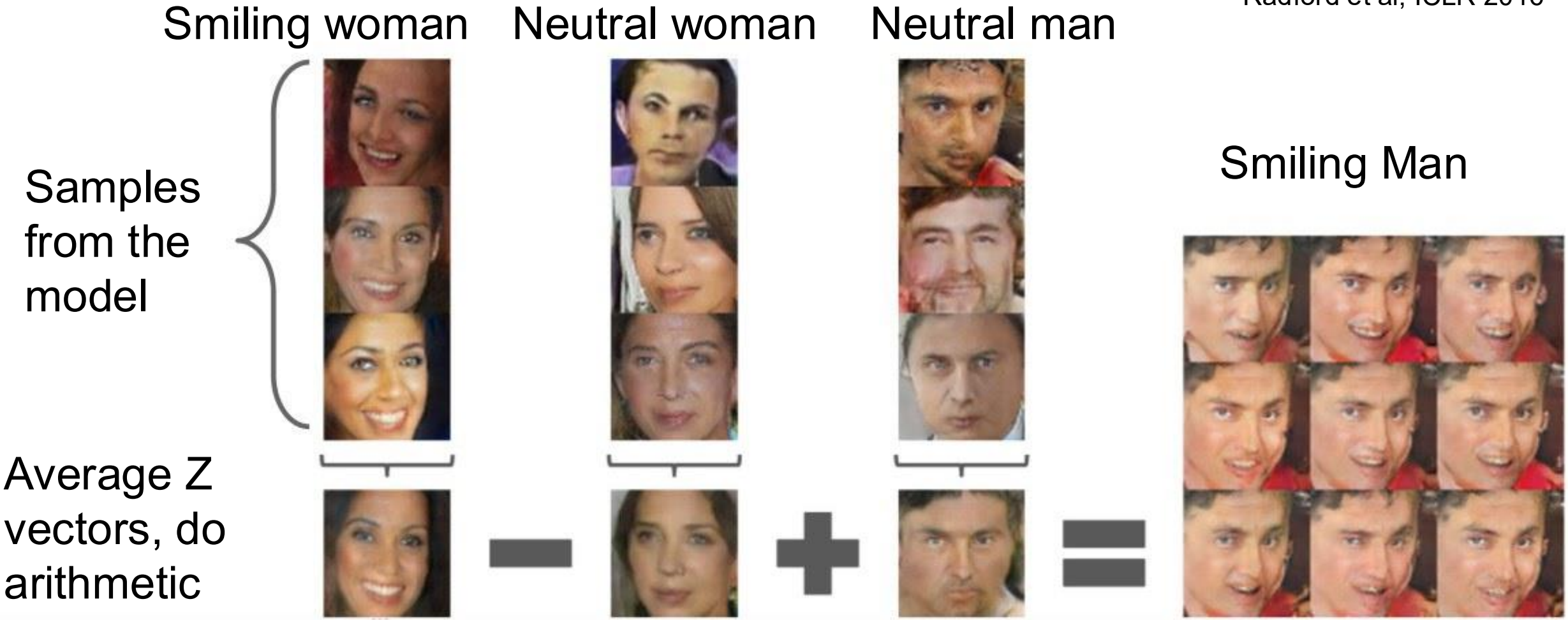
Generative Adversarial Nets: Interpretable Vector Math

Radford et al, ICLR 2016



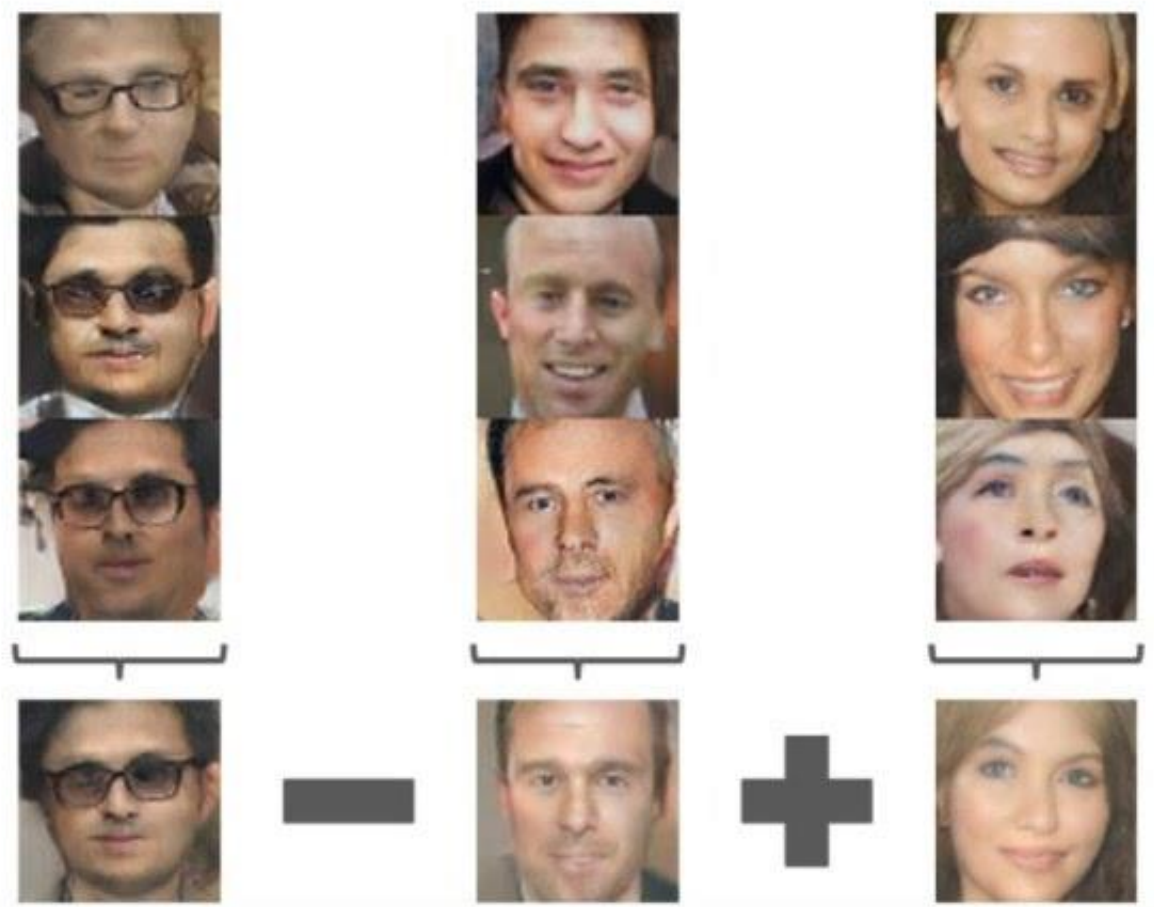
Generative Adversarial Nets: Interpretable Vector Math

Radford et al, ICLR 2016



Generative Adversarial Nets: Interpretable Vector Math

Glasses man No glasses man No glasses woman



Radford et al,
ICLR 2016

Generative Adversarial Nets: Interpretable Vector Math

Radford et al,
ICLR 2016

Glasses man



No glasses man



No glasses woman



Woman with glasses



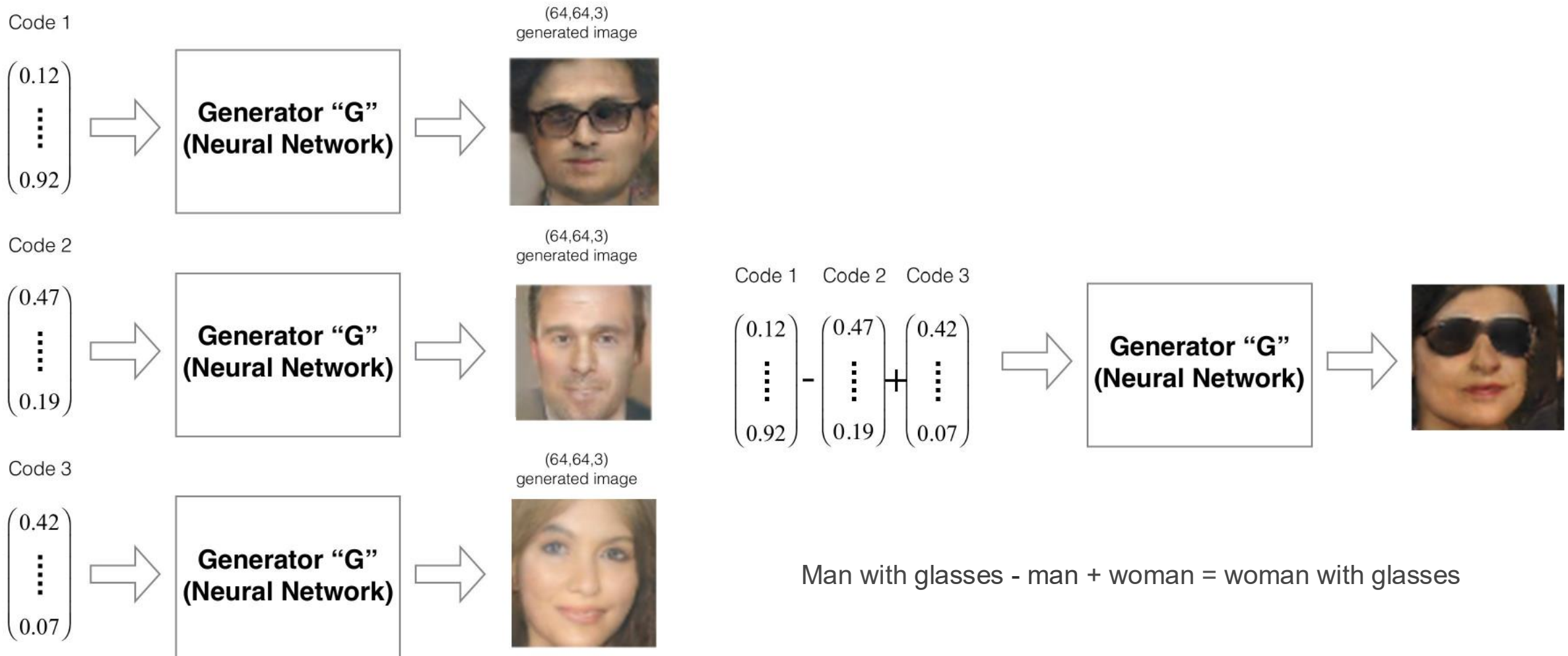
-

+

=

Results

Operation on codes



Style-based transfer: results



2017: Year of the GAN

Better training and generation



(a) Church outdoor.



(b) Dining room.



(c) Kitchen.



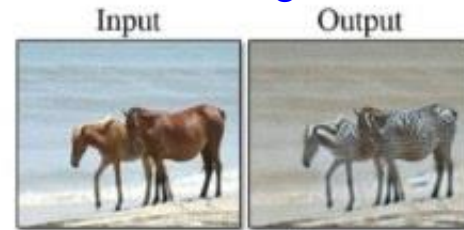
(d) Conference room.

LSGAN. Mao et al. 2017.



BEGAN. Bertholet et al. 2017.

Source->Target domain transfer



horse → zebra



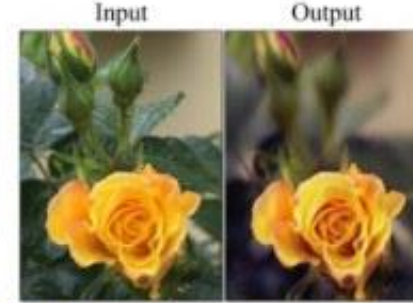
zebra → horse



apple → orange



CycleGAN. Zhu et al. 2017.



Text -> Image Synthesis

this small bird has a pink breast and crown, and black primaries and secondaries.



this magnificent fellow is almost all black with a red crest, and white cheek patch.



Reed et al. 2017.

Many GAN applications

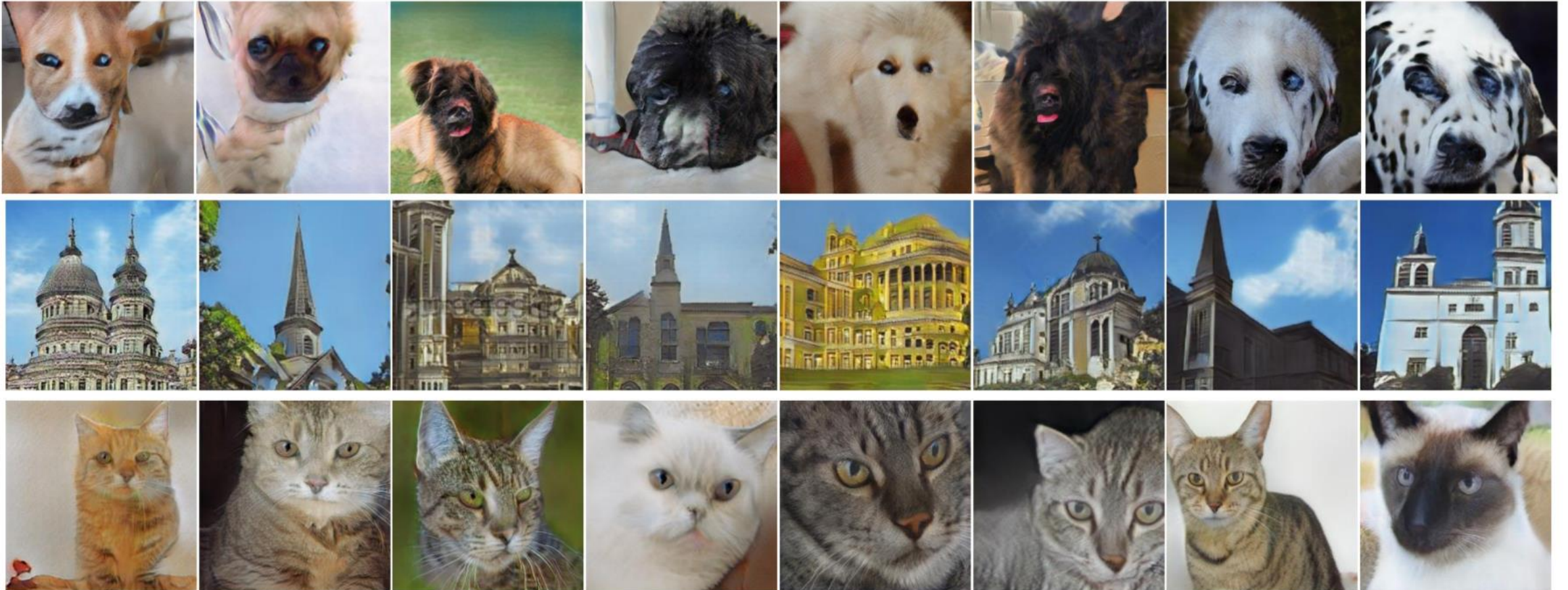


Pix2pix. Isola 2017. Many examples at <https://phillipi.github.io/pix2pix/>

Results

Image Generation:

Samples from the “generated distribution”



[Zhang et al. (2017): StackGAN++]

Frontiers in GAN research



2014



2015



2016



2017



2018

Resources

- **GAN Paper:** Goodfellow, Ian J., et al. "Generative adversarial nets." *Advances in neural information processing systems* 27 (2014).
- https://papers.nips.cc/paper_files/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html
- **Inception Score Paper (Section 4):** Salimans, Tim, et al. "Improved techniques for training gans." *Advances in neural information processing systems* 29 (2016).
- https://proceedings.neurips.cc/paper_files/paper/2016/file/8a3363abe792db2d8761d6403605aeb7-Paper.pdf
- **FID measure Paper:** Heusel, Martin, et al. "Gans trained by a two time-scale update rule converge to a local nash equilibrium." *Advances in neural information processing systems* 30 (2017).
- https://proceedings.neurips.cc/paper_files/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf